

AI-Assisted SDLC Documentation Generator for Automated Software Requirement and Design Artifacts

B-Tech - CE & ICT, Semester-VI

Prepared at



**Bhaskaracharya National Institute for Space Applications & Geo-informatics
Ministry of Electronics and Information Technology, Govt. of India.
Gandhinagar**

Prepared By

Diya Makwana

ID No. 26BISAG0484

Tanisha Diyora

ID No. 26BISAG0485

Guided By:

Dr. Riddhi Thakkar

Department of CE

PDEU, Gandhinagar.

External Guide:

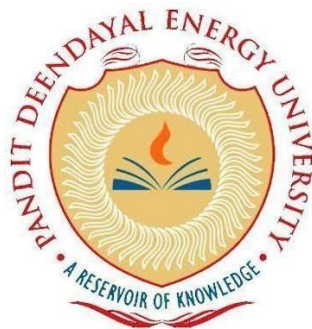
Sidhdharth Patel

ITSA,

BISAG-N, Gandhinagar

SUBMITTED TO

Pandit Deendayal Energy University, Gandhinagar





**Bhaskaracharya National Institute for Space Applications
and Geo-informatics
MeitY, Government of India**

**प्रमाणपत्र
CERTIFICATE**

दिनांक/Date: _____

यह प्रमाणित किया जाता है कि पंडित दीनदयाल एनर्जी यूनिवर्सिटी, गांधीनगर के छठे सेमेस्टर बी.टेक कंप्यूटर इंजीनियरिंग एंड सूचना एवं संचार प्रौद्योगिकी के छात्र सुश्री दिया मकवाना और सुश्री तनिषा दियोरा ने बायसेग-एन में छठे सेमेस्टर की समर इंटरशिप सफलतापूर्वक पूरी कर ली है। उन्होंने 13 मई, 2026 से 13 अगस्त, 2026 तक स्वचालित सॉफ्टवेयर आवश्यकताओं एवं डिजाइन आर्टिफैक्ट्स के लिए एआई-सहायित एसडीएलसी दस्तावेज़ जनरेटर के विकास पर काम किया है।

जहाँ तक हमें सूचना है, यह उनके द्वारा किया गया एक मौलिक और प्रामाणिक कार्य है। संस्थान में अपने कार्यकाल के दौरान, उन्होंने आत्म-प्रेरणा और कर्तव्यनिष्ठा का परिचय देते हुए काम किया। हम उनके उज्ज्वल भविष्य की कामना करते हैं।

This is to certify Ms. Diya Makwana and Ms. Tanisha Diyora, students of 6th Semester B-Tech-CE and ICT respectively from Pandit Deendayal Energy University, Gandhinagar, have satisfactorily completed the 6th semester summer internship at BISAG-N. They have worked on AI-Assisted SDLC Documentation Generator For Automated Software Requirement And Design Artifacts, from May 13th, 2026 to August 13th, 2026.

To the best of our information this is an original and bonafide work done by them. During their tenure at the institute, they demonstrated self-motivation and conscientiousness. We wish them a successful future ahead.

सिद्धार्थ पटेल
Sidhdharth Patel
आईटीएसए
ITSA,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

पुनीत लालवानी
Punit Lalwani
अपर निदेशक सह सीआईएसओ
Additional Director cum CISO,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

Pandit Deendayal Energy University, Gandhinagar



CERTIFICATE

*This is to certify that the 6th Semester Summer Internship Project entitled **"AI-Assisted SDLC Documentation Generator for Automated Software Requirement and Design Artifacts"** has been carried out by Diya Makwana & Tanisha Diyora under my guidance in fulfilment of the degree of Bachelor of Technology in COMPUTER ENGINEERING (6th Semester) of Pandit Deendayal Energy University – Gandhinagar, during the academic year 2025-2026.*

Guide

Dr. Riddhi Thakkar

Head of the Department

Dr. Shakti Mishra

About BISAG-N



Modern day planning for inclusive development and growth calls for transparent, efficient, effective, responsive and low-cost decision-making systems involving multi-disciplinary information such that it not only encourages people's participation, ensuring equitable development but also takes into account the sustainability of natural resources.

The applications of space technology and Geo-informatics have contributed significantly towards the socio-economic development. Taking cognizance of the need of geo-spatial information for developmental planning and management of resources, the department of Ministry of Electronics and Information Technology, Government of India, established "Bhaskaracharya National Institute for Space Applications and Geo-informatics" (BISAG-N).

BISAG-N is ISO 9001, ISO 27001, ISO 19115, ISO 19157 and CMMI: L5 certified national institute. BISAG-N, which was initially set up to carry out space technology applications, has evolved into a center of excellence, where research and innovations are combined with the requirements of users and thus acts as a value-added service provider, a technology developer and as a facilitator for providing direct benefits of space technologies to the grass root level functions/functionaries.



BISAG- N Journey

The journey of Institute started as a state government organization, named as **Remote Sensing and Communication Centre (RESECO)** operating under a limited mandate from 1997 to 2003, focusing on utilization of space technologies such as satellite communication, remote sensing and geospatial applications for supporting various developmental initiatives exclusive for the State of Gujarat. Opinions of various stakeholders at grass root levels was obtained to ensure alignment of Institute's work with the development and welfare need of the society. Numerous sessions of different strata of society were organized including Member of Legislative Assembly (MLAs), village sarpanch, farmers, various skill workers, women and students to obtain insights into what are the needs with simplicity of solutions, and processes of their groups and how technology can fulfil those needs and alleviate any constraints for improving the quality of life.

The basic works of maps generation as per the needs of various state government departments laid the groundwork for the institute's advancement. In December 2003, the name of the institute changed as **BISAG (Bhaskaracharya Institute for Space Applications and Geo-informatics)** with an enhanced mandate to promote satellite communication-based education and skill development facility thereby ensuring increased and more actively involvement in governance processes. The name of the institute was changed to honor the celebrated 12th-century mathematician and astronomer, Bhaskaracharya (1114-1185 AD) who provided in-depth insights about various astronomical and mathematical concepts, formulae and calculations.

The institute's services were extensively utilized by various departments of Gujarat Government under the leadership and patronage of the then Honorable Chief Minister Shri Narendrabhai Modi. Numerous times the assembly meetings were held in BISAG-N premises to amalgamate governance and technology for the well-being of the state. The state leadership expressed to the ministers and bureaucrats on multiple occasions the importance of visualization of development plans and schemes on maps and thereby harnessing the capabilities of BISAG as a **"go-to place"** for various technological and geo-informatics solutions required by concerned ministries and departments for the benefit of citizens and development of the state.

On one of such occasions in 2004, Honorable Chief Minister Shri Narendra Modi said, "Friends, I can clearly visualize that within a span of five years, all streams of government administration shall converge into BISAG. This institute shall be a huge treasure of information, if any decisions are to be taken or any department require any information or need any policy change; the **'Source of Information'** shall be BISAG. There for earlier we can transform BISAG into a **'Source of Information'** and position it as back-bone of all governance and administration activities, and the sooner the benefits of science and technology shall be available to the society."



Since then, inspired by above and gaining insights while working collaboratively for various development initiatives of central and state government department, the Institute is focused on following vision:

"To empower and serve the nation through development of space and geo-spatial technologies, its applications, solutions and services in accordance to governance principles for socio-economic welfare of the society."

The Institute develops geo-spatial technologies, databases, software and processes along with skilled resources to provide visualization, analysis and decision support system to various state government

departments. Also, the Satellite Communication infrastructure was enhanced to add more channels and developed in house expertise to install, operate and maintain various electronic instruments and equipment. The institute constantly ensured adoption of national and international standards as well as best practices to develop and reinforce the solutions delivery process. With the improved confidence and trust of the state government to utilize geo-spatial and satellite communication services for developmental, social welfare and disaster management areas, the institute was supported by the Government of Gujarat to construct dedicated buildings for ensuring services are provided at even larger scale. This expansion broadened the institute's scope to integrate advanced techniques such as managing large databases, mathematical modelling, enabling prediction and simplified architecture for the solution aligning with the government priorities for improving efficiency, transparency and pace of development

Along with providing the e-governance support to Gujarat State, the institute also started supporting various other states and central government ministries as and when required for map and decision support system development services. Recognizing this potential, the institute's mandate was further enhanced to cater to the geo-spatial and satellite communication-based e-governance services to all central government ministries as well as various states and union territories as a **National** institute. The name of the institute was then changed as Bhaskaracharya **National** Institute for Space Applications and Geo-informatics (**BISAG-N**), registered as an Autonomous Scientific Society, as per the Societies Registration Act, 1860, under the Ministry of Electronics and Information Technology (MeitY), Government of India (GoI). BISAG-N; enhanced mandate includes to undertake technology development & management; research & development; facilitate national and international cooperation and capacity building; support technology transfer and entrepreneurship development in area of geo-spatial technology.

Today, BISAG-N is collaboratively working with 50+ central government ministries and most of the states and union territories for supporting various developmental programs and citizen centric services. BISAG-N provides the technological support to these stakeholders from its Gandhinagar and New Delhi facilities. These facilities have adequate skilled personnel and state-of-the-art infrastructure for providing geo-spatial solutions and software applications including support of satellite communication-based education and training. Thus, BISAG-N is playing pivotal role as a one stop center for providing geo-spatial and space technologies for various flagship programme being undertaken for the development of nation in line with geo-spatial education, logistics and various other policies and program of central and state governments.



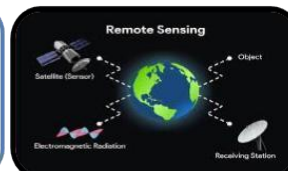


Satellite Communication:

For promotion and facilitation of the use of broadcasting facility for distant interactive training, education, skill development, health, agriculture and extension

Remote Sensing:

For inventory, mapping, developmental planning and monitoring of natural and built resources



Geographic Information System:

For conceptualization, creation and organization of multi-purpose common digital databases for development of sectoral/integrated decision systems.

Global Navigation Satellite System:

For location based services, engineering applications and research

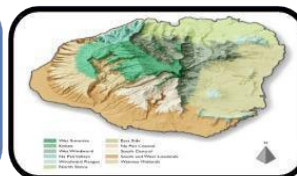


Photogrammetry:

For creation of maps, Digital Elevation Model (DEM), terrain characteristic maps for resource planning and management

Cartography:

For Thematic Mapping and generation of value added maps



Software Applications Development:

For wider usage in geo-spatial applications, Decision Support Systems (DSS) and ERP solutions.

Innovation, Research & Development:

Quantum Computing, Digital Twins, Cyber Security, Block Chain, Big Data, AI/ML

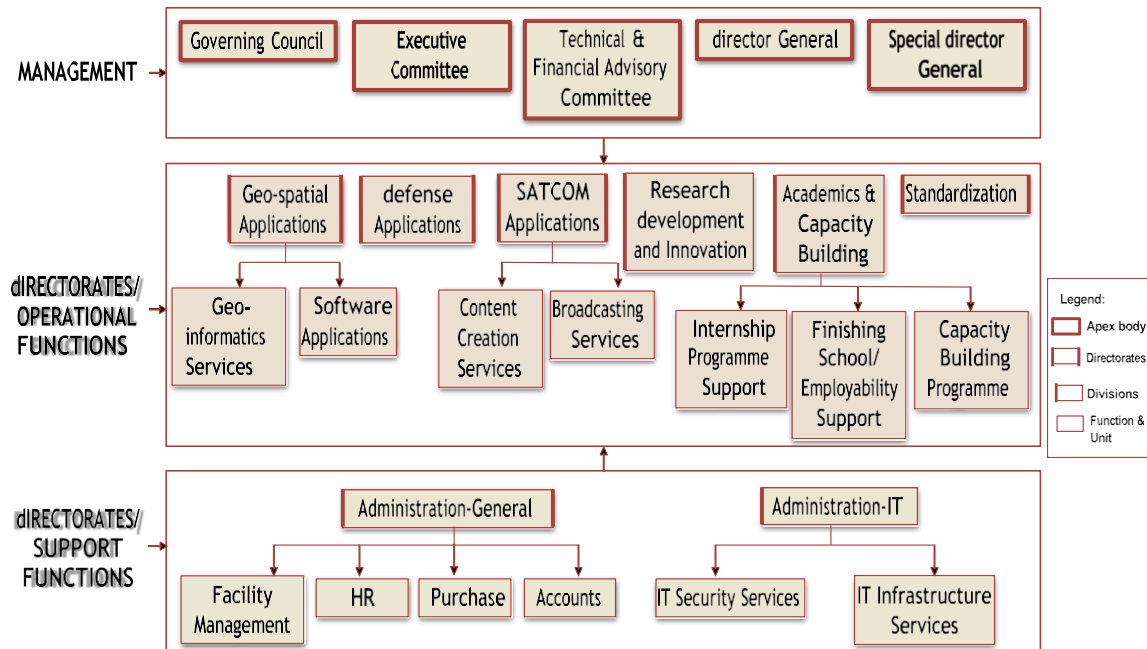


Education and Training:

For providing Education, Training & Technology knowledge transfer to large number of students, end users and collaborators for promoting Research & Development

Organizational Setup

BISAG-N main domain areas include Satellite Communication, Geo-spatial Solutions and Technology Development, Geo-informatics Applications Development, Standardization, Academics & National/ International Co-operation and Administration.



BISAG-N: Functions and Divisions

SATCOM Applications

The SATCOM Directorate provides services of distance education, capacity building, training, extension, awareness and empowerment using the state-of-the-art facility comprises of an uplink earth station, control room, studios and a satellite communication-based network. The services are primarily utilized by Ministry of Education and various other central and states ministries. Through Ministry of External Affairs (MoEA), BISAG-N is now poised to provide SATCOM based education to neighbouring countries also.

SATCOM operations are inspired by the wisdom of Hon'ble Prime Minister of India as elicited on numerous occasions. For example, in 2004 as Hon'ble Chief Minister of Gujarat, Shri Modi explained the importance of learning through the ancient way of visualization using virtual media with these words:

“The way Eklavya studied by making a statue, why we can't study with the help of a T.V.? Yes, we can. Was any cinema song ever taught to us in any class? Was it ever taught by any teacher? We learnt song from T.V., isn't it? If we have learnt entire cinema song, learnt every word, learnt scale, learnt rhythm, it means we can learn through T.V. also.”



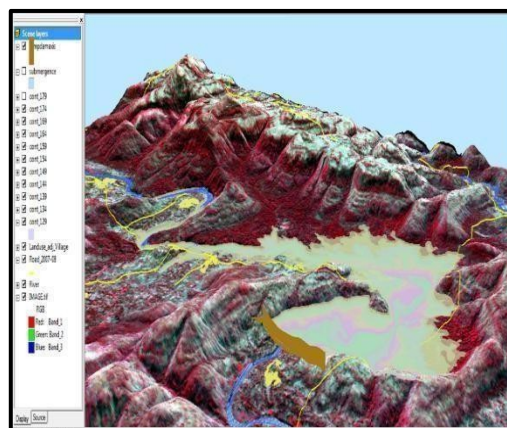
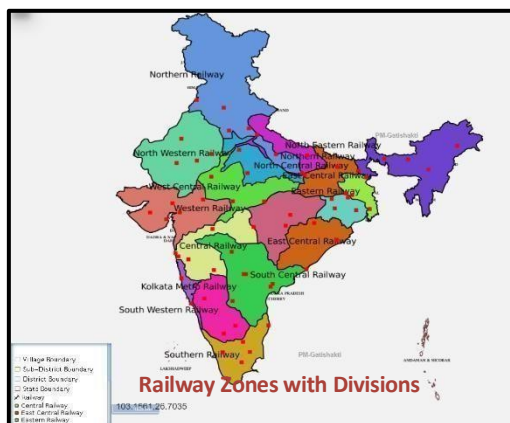
There are twelve state-of-the-art studios for live recording of content supported by dedicated editing labs, subtitling, translation, animation and various other aspects. The institute has set up ten studios at Gandhinagar facility and another two studios at New Delhi. Infrastructure for transmission and Ku band uplink teleports (11m/9.3m) of more than 350 channels is installed, integrated and maintained by inhouse team. BISAG-N support various initiatives of Ministry of Education, Government of India, as well as other central and state government ministries such as PM e-Vidya, Swayam Prabha etc. as detailed further.



Geo-spatial Applications Directorate

BISAG-N's Geo-Spatial Applications Directorate provides specialized services and solutions to implement Geographic Information Systems (GIS) for providing impetus to planning and developmental activities at grass root level. The objective of GA directorate is to provide analysis and decision support services to various stakeholders through scientifically organized, comprehensive, multi-purpose, compatible and large scale geo-spatial databases.

Geo-spatial Applications Directorate creates and organizes large-scale and multi-purpose geospatial datasets in collaboration with central and state government departments and agencies. A well-qualified team of GIS technicians and engineers ensure that the datasets are compatible, standardized and authenticated for use in analysis, planning and decision-support functions. Goals and roles of converging organizations are taken into consideration while geo-spatial database creation and software development activities



Software Development Division

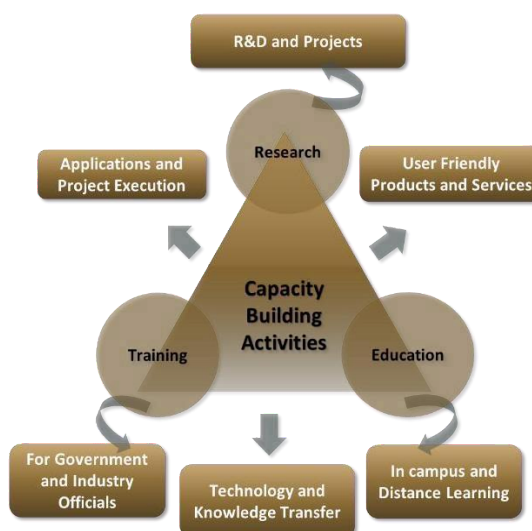
The geo-spatial solutions are packaged and provided as software products for ease of retrieval of spatial and feature attributes data, enabling wide range of analysis, prediction and decision support services. Software technologies like Java, Python, JavaScript, Spring Boot, React JS, Node JS, Microservices, XML, GeoJSON, GeoServer, QGIS, Android and iOS frameworks are used to develop user-friendly and reliable solutions.



Academy of Geo-informatics

BISAG-N, being a national institute for geo-spatial and emerging technology-based applications, has a strong focus on research, development, capacity building and employability support for under-graduate, post-graduate and doctoral students. The institute's infrastructure and supportive work environment provide an ideal platform for students pursuing thesis work in a variety of technologies and domains.

BISAG-N is equipped with cutting-edge infrastructure to support research across multiple disciplines. Dedicated laboratories for research on the application of artificial intelligence, machine learning, GIS (Geographic Information Systems), remote sensing, quantum computing, IoT (Internet of Things) and other emerging technologies are established high-performance computing facilities for intensive data processing tasks are available in the institute. Open-source platforms and in-house developed software are available for projects, research and innovation activities. More than 300 students are supported every year for learning practical based aspects of geo-spatial and emerging technology applications while completing their projects.



BISAG-N Finishing School Programme (FSP) supports wards of special groups of society such as scheduled tribe, scheduled caste, non-gazetted security, armed forces citizens and martyr etc, for development of technical and soft skills to improve their employability including support for entrepreneurship and employment opportunities.

The school provides basic and specialized training to help participants develop capacities as required by the for related industry/domain. The participants undergo thorough counselling sessions as well as mock/actual interviews for improving employability in relevant industries as per the interest of the candidates. Along with the skills training the participants are also allowed to observe engaged in ongoing projects of organization. BISAG-N academic activities also provide capacity building support to government officials for effectively using geo-spatial solutions and decision support systems for performing planning and monitoring for various government schemes and initiatives.

Industrial visits of school students are organized to provide them exposure of space applications and geo-informatics thereby supporting the development of scientific temperament at early stage. More than 5000 students visit the premises every year to get exposure of the activities being carried out at BISAG-N.



Standardization Directorate

BISAG-N has a dedicated directorate for ensuring continuously focus on innovation, standardization and institutionalization of solutions & service. The Standardization Directorate focuses on implementation of various national and international standards related to program solutions and data management. The directorate develops internal SOPs and policies for compliance to these standards and provides training for effective implementation. The directorate also liaison with the various departments and external agencies during internal and external audits. In addition, the directorate acts as member in standards committees of Bureau of Indian Standards (BIS) and other standard developing bodies for development of standards.

Goals of Standardization Directorate include:

- To proactively implement product and process improvements initiatives using various national & international standards and best practices. This includes ensuring quality and reliability of geo-spatial data, completeness, updation of metadata, functional performance and security reviews of applications, capacity building support to end-users for adoption of solutions and information security-oriented culture development
- To develop a culture of efficiently working as per standards while fostering innovation
- To perform reviews, gap analysis, training, documentation and standards implementation support activities

Administration Directorate

Administrative Directorate provides strategic and management support to various activities of BISAG-N. The directorate ensures infrastructure and facilities management in addition to providing services such as coordination during visits of various delegations, logistic support to BISAG-N staff during travel, accounting and finance, procurement, training, safety and security, legal compliance, maintenance, housekeeping, horticulture, construction and extension projects, campus monitoring, canteen management as well as celebrations and welfare activities. The directorate also ensure recruitment and human resource development as per the needs of various projects and directorates while upholding principles of equality, fairness and diversity in work force.

BISAG-N has adequate human resources, well trained for behavioral, functional and domain competencies, with clear roles and responsibilities in alignment with service delivery expectations. The staff well-being and motivation initiatives are ensured for maintaining a culture of shouldering responsibilities proactively and delivering high-performance utilizing multi-disciplinary knowledge.

Administration Directorate ensure strategic and compliance support to operations of the institute by managing human resources, facilities, expansion projects, finance and governance aspects. The directorate also coordinates with Council stakeholders and members of Governing and Executive committee as well as other high-level committees.

Defence Applications Directorate

This directorate develops space, geo-spatial, AI and other emerging technologies-based applications under the guidance of various agencies of the Ministry of Defence, Government of India. All the technologies used are developed inhouse of customized open source. Considering the strategic significance of these projects, dedicated team and premises are allocated for ensuring information security. Most of the critical activities and data management are performed directly at various defence establishments.

Besides the projects of various infrastructure and social welfare ministries and departments of central and state government, BISAG-N has a dedicated directorate for supporting defence and security departments for providing data products, decision support systems and services involving geo-spatial and various other emerging technologies. The projects are executed in high security dedicated laboratory for various units of army, navy and air-force for supporting strategic and operational aspects of the operations.

Through its innovative geospatial solutions and technological expertise, BISAG-N is playing a crucial role in enhancing the nation's defence capabilities and ensuring long-term strategic autonomy. Due to nature of high secrecy of these strategic projects, they are not included in details in this compilation.

Research and Development

BISAG-N's research and development group supports all other directorates for providing solutions related to new definitions and technology. The group develops innovative applications, features, tools and services utilizing technologies such as remote sensing, photogrammetry, artificial intelligence, machine learning, internet of things, computer vision, big data and quantum computing, which are later integrated in live projects.

BISAG-N has made significant progress for the development of Artificial Intelligence/ Machine Learning (AI/ML) applications. Algorithms and models have been developed which can be customized and used with training, validation and live data upon their data for accurate predictions and smarter results.

The applications developed by the R&D team include support for the pluggable APIs which can use the machine learning with minimal code reconfiguration. Various GIS applications are developed at BISAG-N. Utilizing spatial and non-spatial processing of Big Data and Web GIS architecture These applications support multi-terabyte datasets and large number of consumers. Big Data applications for visualization developed by the institute has enabled several projects to explore and analyse multiple large archives. Through continuous research and development, BISAG-N's R&D capabilities are continuously enhanced. This allows to enhance users' satisfaction, growth and success stories.

Cyber Security Cell

Cyber Cell reviews and update the threat landscape for the BISAG-N's critical infrastructure on a regular basis in line with various vulnerabilities detected and published by reputed repositories. The cell verifies the applications using Vulnerability Assessment & Penetration Testing (VAPT) ensuring the compliance to Guidelines for Indian Government Websites (GIGW), Open Web Application Security Project (OWASP), SANS 25 etc. The cell also ensures implementation of good information security practices in line with ISO 27001 such as serious logs review, vulnerability analysis, exception reporting, maintenance of firewall and other network monitoring tools.

CANDIDATE’S DECLARATION

We declare that the 6th semester internship project report entitled “AI-Assisted SDLC Documentation Generator For Automated Software Requirement And Artifacts” is our own work conducted under the supervision of the external guide **Mr. Sidhharth Patel** from BISAG-N ([Bhaskaracharya National Institute for Space Applications & Geo-informatics](#)). We further declare that to the best of our knowledge, the report for this project does not contain any part of the work which has been submitted previously for such project either in this or any other institutions without proper citation.

Candidate 1’s Signature

Diya Makwana

Student ID: 26BISAG0484

Candidate 2’s Signature

Tanisha Diyora

Student ID: 26BISAG0485

Submitted To:

Pandit Deendayal Energy University – Gandhinagar

ACKNOWLEDGMENT

We are grateful to **Shri T.P. Singh, Principal Advisor & Executive Vice Chairman, Governing Council** and **Dr. Vinay Thakur, Director General (BISAG-N)**, for giving us this opportunity to work the guidance of renowned people of the field of GIS-Based Portal, also providing us with the required resources in the company.

We would like to express our endless thanks to our external guide, **Mr. Sidhdharth Patel**, and to the Training Cell, **Mr. Punit Lalwani & Mr. Sidhdharth Patel** at Bhaskaracharya National Institute of Space Application and Geo-informatics, for their sincere and dedicated guidance throughout the project development.

Also, our hearty gratitude to our Head of Department, **Dr. Gaurav Ameta** and our internal guide **Prof. Arpita Limbachiya** for giving us encouragement and technical support on the project.

Diya Makwana.

Student ID: 26BISAG0484

Tanisha Dyora.

Student ID: 26BISAG0485

ABSTRACT

The AI-Assisted SDLC Documentation Generator for Automated Software Requirement and Design Artifacts is a web-based platform designed to automate the creation of software development lifecycle (SDLC) documentation from project requirements. The system addresses the time-consuming and repetitive process of manually preparing software engineering artifacts by allowing users to provide project requirements once and generating structured documentation based on the selected SDLC methodology.

The proposed system uses Artificial Intelligence and Large Language Models (LLMs) to analyze project requirements and generate various software engineering artifacts, including Software Requirements Specification (SRS), High-Level Design (HLD), Low-Level Design (LLD), Work Breakdown Structure (WBS), Test Cases, User Stories, API Documentation, Database Schema, and UML diagrams. The platform supports multiple development methodologies, including Agile, Waterfall, V-Model, Spiral, RAD, and DevOps, enabling documents to be generated according to the selected methodology.

The system follows a microservices-based architecture, with React.js used for the frontend, Node.js and Express.js for backend services, FastAPI for AI-related processing, and MongoDB for data management. AI-generated outputs are structured and formatted into professional documentation, while Mermaid.js is used for generating and visualizing UML diagrams. The platform also provides document viewing and export capabilities.

The proposed solution aims to improve the consistency, efficiency, and organization of SDLC documentation while reducing repetitive manual effort. It provides a centralized platform for transforming project requirements into structured software engineering artifacts and demonstrates the practical application of AI in software development processes.

CONTENTS

Chapter No.	Description	Page No.
-	List of Figures	I
-	List of Tables	II
-	List of Abbreviations	III
-	List of Definitions	IV
-	List of Screenshots	V
1	Introduction	1
1.1	Project Details	1
1.2	Purpose	1
1.3	Scope	2
1.4	Objectives	3
1.5	Technology Stack Used	4
1.6	Literature Review	4
2	Project Management	7
2.1	Feasibility Study	7
2.1.1	Technical Feasibility	7
2.1.2	Time Schedule Feasibility	8
2.1.3	Operational Feasibility	9
2.1.4	Implementation Feasibility	9
2.2	Project Planning	10
2.2.1	Development Approach & Justification	10
2.2.2	Milestones & Deliverables	11
2.2.3	Roles & Responsibilities	13
2.2.4	Group Dependencies	13
2.3	Project Scheduling (Gantt/PERT chart)	16
3	System Requirement Study	17
3.1	Existing System Overview	17
3.2	Limitations of the Existing System	18
3.3	User Characteristics	19
3.4	Functional Requirements	20
3.5	Non-Functional Requirements	21
3.6	Hardware & Software Requirements	23
3.7	Constraints	23
3.7.1	UI Constraints	24
3.7.2	Communication Interface	24
3.7.3	Hardware Interface	24
3.7.4	Criticality of Application	24
3.7.5	Safety & Security Considerations	25
3.8	Assumptions & Dependencies	25
4	Proposed System Requirements	27
4.1	Overview of Proposed System	27
4.2	Module Descriptions	27
4.2.1	Module A Project Lifecycle	27
4.2.2	Module B Organizational Knowledge Base	27
4.2.3	Module C Organizational Training	28
4.3	System Features	28
4.4	Advantages of Proposed System	30
5	System Design	31
5.1	System Architecture Design	31
5.2	UML Diagrams	31

Chapter No.	Description	Page No.
5.2.1	E-R Diagram	32
5.2.2	Use Case Diagram	33
5.2.3	Class Diagram	34
5.2.4	Sequence Diagram	34
5.2.5	Activity Diagram	35
5.2.6	DFD Diagram	35
5.2.7	Deployment Diagram	36
5.3	Database Design	37
5.3.1	Table Design & Relationships	37
5.3.2	Normalization	38
5.3.3	Data Dictionary	39
5.4	GUI Design	43
5.5	Screenshots	45
6	Implementation Planning	48
6.1	Implementation Environment	48
6.2	Tools & Technologies Used	50
6.3	Coding Standards Followed	51
6.3.1	Project & Folder Structure	52
6.3.2	Naming & Formatting Conventions	52
6.3.3	Error Handling & API Responses	53
6.3.4	Documentation & Comments	53
7	Testing	54
7.1	Testing Objectives & Strategy	54
7.2	Levels of Testing	55
7.2.1	Unit-Level Testing	56
7.2.2	Integration Testing	56
7.2.3	System / Functional Testing	56
7.2.4	SDLC-Model & Terminology Verification	57
7.2.5	UML Diagram Verification	57
7.2.6	Regression Testing	57
7.3	Test Cases	57
7.4	Bug Tracking & Resolution	58
8	Limitations and Future Scope	60
8.1	Limitations	60
8.2	Future Scope	62
9	Conclusion and References	64
9.1	Conclusion	64
9.2	References	65
10	Appendices	66

List of Figures

Figure No.	Figure Description	Page No.
Figure 1.3	Three-Module System Scope Overview	3
Figure 2.2.4	Module Dependency Diagram	15
Figure 2.3	Gantt Chart of the Project Schedule	16
Figure 3.1	Existing (Manual) Process vs. Proposed (Automated) Process	17
Figure 4.3	System Features Map	29
Figure 5.1	System Architecture Diagram	31
Figure 5.2.1	Entity-Relationship (ER) Diagram	32
Figure 5.2.2	Use Case Diagram	33
Figure 5.2.3	Class Diagram	34
Figure 5.2.4	Sequence Diagram — Generate/Review/Fix Cycle	34
Figure 5.2.5	Activity Diagram — Project Approval to Document Pass	35
Figure 5.2.6 (i)	Data Flow Diagram — Level 0 (Context)	36
Figure 5.2.6 (ii)	Data Flow Diagram — Level 1	36
Figure 5.2.7	Deployment Diagram	36
Figure 5.2.8	Component Diagram	37
Figure 5.2.9	Document Status State Diagram	37
Figure 6.1	Local Development Setup / Service Ports Diagram	49
Figure 7.2	Levels of Testing (Testing Pyramid)	55
Figure 8.2	Future Scope Roadmap (Indicative Priority Order)	63

List of Tables

Table No.	Table Description	Page No.
Table 1.1	Project Details (Title, Organization, Role, Duration, Team Size, Mentor)	1
Table 1.5	Technology Stack Used in the Proposed System	4
Table 1.6	Comparison with Existing Tools	5
Table 2.1	Feasibility Study Summary	7
Table 2.2.2	Project Milestones and Deliverables	11
Table 2.2.4	Module Dependency Matrix	14
Table 3.2	Limitations of the Existing System	18
Table 3.3	User Characteristics	19
Table 3.4	Functional Requirements (numbered)	21
Table 3.5	Non-Functional Requirements of the Proposed System	21
Table 3.6	Hardware & Software Requirements	23
Table 3.7	Constraints Summary	23
Table 3.8	Assumptions & Dependencies	26
Table 4.2	Modules and Functions of the Proposed System	28
Table 4.4	Advantages of the Proposed System	30
Table 5.2	UML Diagram Types & Purpose	32
Table 5.3.1	MongoDB Collections & Relationships	38
Table 5.3.2	Denormalization / Embedding Decisions	38
Table 5.3.3	Data Dictionary (documents, projects, issues, master_data_entries, execution_documents, risk_repository, historical_estimates, employees, training_programs, training_sessions)	39
Table 6.1	Implementation Environment	48
Table 6.2	Tools & Technologies Used	50
Table 6.3.2	Coding Standards Followed	53
Table 7.2	Levels of Testing Summary	55
Table 7.3	Test Cases and Results	57
Table 8.1	Limitations Summary	60
Table 8.2	Future Scope Summary	63
Table B.1	Appendix B: Environment Variables Reference	67
Table C.1	Appendix C: Supported SDLC Models Reference	67
Table D.1	Full Document Type List by Module	68
Table E.1	Sample Field-to-Requirement Mapping	69

List of Abbreviations

Abbreviation	Full Form
SDLC	Software Development Life Cycle
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
MERN	MongoDB, Express.js, React.js, Node.js
ODM	Object Data Modeling
JSON	JavaScript Object Notation
HTTP / HTTPS	HyperText Transfer Protocol (Secure)
JWT	JSON Web Token
RBAC	Role-Based Access Control
UML	Unified Modeling Language
ER / E-R	Entity-Relationship
DFD	Data Flow Diagram
GUI	Graphical User Interface
UI	User Interface
SRS	Software Requirements Specification
HLD	High-Level Design
LLD	Low-Level Design
SDD	Software Design Document
WBS	Work Breakdown Structure
NFR	Non-Functional Requirement
RAD	Rapid Application Development
CRUD	Create, Read, Update, Delete
PDF	Portable Document Format
RAM	Random Access Memory
GPU	Graphics Processing Unit
VPS	Virtual Private Server
VPN	Virtual Private Network
LAN	Local Area Network
TLS	Transport Layer Security
URL	Uniform Resource Locator
PMBOK	Project Management Body of Knowledge
PERT	Program Evaluation and Review Technique
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission

List of Definitions

Term	Definition
SDLC (Software Development Life Cycle)	The structured sequence of stages a software project passes through, from requirements to deployment and maintenance.
Waterfall / Agile-Scrum / Agile-Kanban / V-Model / Spiral / RAD / DevOps	The seven SDLC methodologies the platform supports, each with its own terminology, structure, and required document fields.
Microservices Architecture	A design approach where an application is built as a set of independently deployable services (here: api-gateway, document-service, requirement-service, ai-service) that communicate over REST APIs.
API Gateway	The service that routes incoming requests to the appropriate backend service (document-service, requirement-service, or ai-service).
MERN Stack	A web development stack composed of MongoDB, Express.js, React.js, and Node.js.
Mermaid.js	A JavaScript library used by the platform to automatically render UML and flow diagrams from generated structure.
Generate-Review-Fix Cycle	The platform's core workflow: a document is generated, scored and reviewed against SDLC-model rules, and then iteratively fixed rather than rewritten from scratch.
verification.py	The code-level module that checks a proposed fix against SDLC-model rules before accepting it, rather than trusting a re-prompted AI result.
Implementation Drift	A defect pattern where generated or edited content deviates from what was actually specified, observed during both platform development and the user-manual writing process.
Opportunity Sheet	The Stage 0 approval-gate document that must be completed before a project unlocks Stage 1 document generation.
Master Data List	An automatically maintained register tracking every document generated for a project, its version, and its current status.
Module A — Project Lifecycle	The platform module covering a single project's documentation from initial approval (Stage 0) through closure (Stage 4).
Module B — Organizational Knowledge Base	The cross-project module storing a Risk Repository and historical cost/effort estimates, surfaced to new projects only as optional, non-automatic suggestions.
Module C — Organizational Training	The workforce-training module, independent of individual projects, covering skill assessment, training delivery, and evaluation reporting.
Risk Repository	Module B's store of risks encountered on past projects and the mitigations that worked for them.
Data Integrity (as used in this project)	The requirement that the system never fabricate content for fields it lacks data for, presenting them as blank templates instead.
MERN Stack	A web development stack composed of MongoDB, Express.js, React.js, and Node.js.
Mermaid.js	A JavaScript library used by the platform to automatically render UML and flow diagrams from generated structure.
Generate-Review-Fix Cycle	The platform's core workflow: a document is generated, scored and reviewed against SDLC-model rules, and then iteratively fixed rather than rewritten from scratch.
verification.py	The code-level module that checks a proposed fix against SDLC-model rules before accepting it, rather than trusting a re-prompted AI result.

List of Screenshots

Screenshot No.	Screenshot Description	Page No.
Screenshot 5.4 (i)	Stage 0 — Opportunity Sheet Approval Screen (Wireframe)	43
Screenshot 5.4 (ii)	Module B — Suggestion Accept / Edit / Reject Screen (Wireframe)	44
Screenshot 5.5 (i)	Project Creation Wizard showcasing dynamic fields adapting to the selected SDLC model	45
Screenshot 5.5 (ii)	Document Review Screen illustrating the split-pane view with AI-generated markdown on the left and the code-verified issue list on the right	46
Screenshot 5.5 (iii)	Stage 3 Execution Dashboard displaying recurrence-based documentation management	46
Screenshot 5.5 (iv)	Module C Training Interface (skill assessment / session logging view)	47
Screenshot F.1 (i)	Stage 3 Execution Dashboard (Execution & Communication Logs)	70
Screenshot F.1 (ii)	Stage 3 Execution Dashboard (Evolving Project Artifacts)	70

1 Introduction

This project addresses the problem of manual, inconsistent SDLC documentation by proposing an automated, methodology-aware generation platform. The following sections cover the project's purpose, scope, objectives, technology stack, and its position relative to existing tools and literature in this domain.

1.1 Project Details

Attribute	Detail
Project Title	AI-Assisted SDLC Documentation Generator For Automated Software Requirement And Design Artifacts
Organization	BISAG-N (Bhaskaracharya National Institute for Space Applications and Geo-informatics), Gandhinagar
Role	MERN Stack Development Intern
Duration	13 th May 2026 to 13 th Aug 2026
Team Size	2
Mentor / Guide	Sidhdharth Patel ()

Table 1.1: Project Details (Title, Organization, Role, Duration, Team Size, Mentor)

1.2 Purpose

The project was undertaken as part of a MERN Stack Development internship at BISAG-N, where the day-to-day work involved functioning in a full-stack software developer capacity. The system was conceived, designed, and iteratively developed and tested across a defined internship period, with continuous review and refinement guided by the project mentor.

Software Development Life Cycle (SDLC) documentation requirement specifications, design documents, test plans, project management artifacts, and organizational records is essential to the successful planning, execution, and governance of any software project. In practice, however, producing this documentation is a time-consuming, repetitive, and error-prone manual activity. Teams following different SDLC methodologies (Waterfall, Agile-Scrum, Agile-Kanban, V-Model, Spiral, RAD, and DevOps) must additionally ensure that each document uses the correct terminology, structure, and level of detail appropriate to their chosen methodology a constraint that is easy to violate when documents are drafted by hand or copied from generic templates.

The purpose of this project is to design and build a platform that automates the generation of SDLC documentation directly from a structured description of a software project its chosen SDLC model, features, modules, technology stack, team roles, and constraints and to make that documentation iteratively refinable through a review-and-fix cycle, rather than requiring documents to be rewritten from scratch whenever an issue is found.

1.3 Scope

The scope of the system spans the complete lifecycle of a software project as well as organization-level knowledge that persists across projects. Specifically, the platform is organized into three modules:

- 1.3.1 Module A Project Lifecycle: documentation generated and maintained for a single project, from an initial opportunity assessment (Stage 0) through SDLC document generation (Stage 1), planning refinement (Stage 2), recurring execution-phase documentation (Stage 3), and project closure (Stage 4).
- 1.3.2 Module B Organizational Knowledge Base: a cross-project store of risk history and historical cost/effort data that new projects can draw on as suggestions, without any data being applied automatically.
- 1.3.3 Module C Organizational Training: a separate workforce-training subsystem covering skill assessment, training program identification, delivery logging, and evaluation — independent of any single software project.

In total, the system's scope covers 41 distinct document types across these three modules. The scope explicitly excludes providing legal, contractual, or compliance sign-off — the platform produces draft-quality, review-ready documentation intended to be checked and approved by project stakeholders, not documentation that bypasses human review.

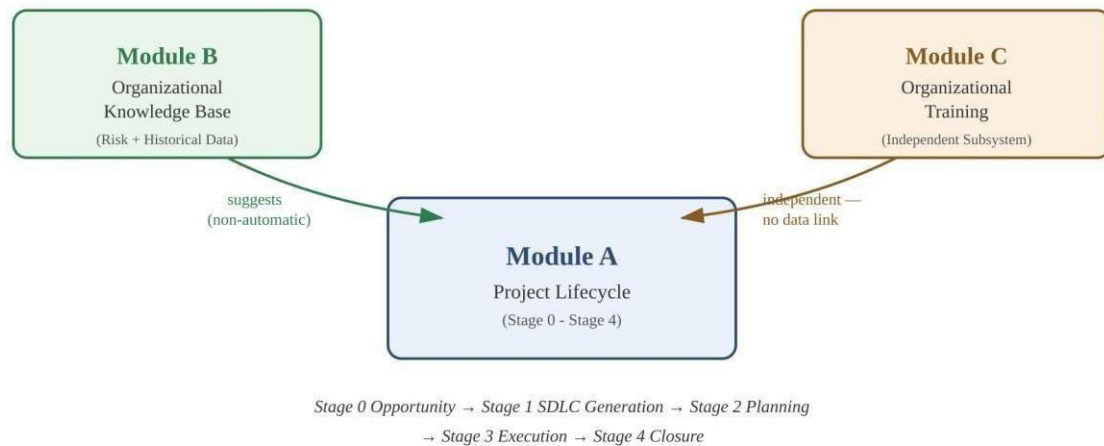


Figure 1.1: Three-Module System Scope Overview

Figure 1.3: Three-Module System Scope Overview

1.4 Objectives

- 1.4.1 To automate the generation of SDLC documents from structured project inputs, reducing the manual effort typically required to produce them.
- 1.4.2 To support multiple SDLC methodologies (seven in total) with methodology-specific terminology, structure, and required fields, rather than a single generic template.
- 1.4.3 To provide an iterative review-and-fix cycle so that documents can be corrected and regenerated without being rewritten from scratch.
- 1.4.4 To prevent fabricated or invented content by ensuring the system only fills in data it has been given, and produces blank templates for information that must come from the user.
- 1.4.5 To build a shared organizational knowledge base that lets new projects benefit from risk and estimation history recorded on past projects.
- 1.4.6 To extend documentation automation beyond individual projects into organization-wide workforce training records.

1.5 Technology Stack Used

The system is built as a set of independently deployable services rather than a single monolithic application, reflecting the separation between the project-facing document generation logic and the underlying AI-assisted generation engine.

Layer	Technology
Frontend	React
Application services	Node.js microservices — document-service, api-gateway, requirement-service
AI/ generation service	Python, FastAPI
Database	MongoDB
Fix verification	Code-level verification gates (verification.py) run within retry loops
Document export	PDF export, Document export

Table 1.5: Technology Stack Used in the Proposed System

The frontend, built in React, hosts the project creation wizard, document review screens, the Stage 3 execution dashboard, and the Module C training interface. The api-gateway routes requests to the appropriate backend service; document-service and requirement-service handle document and requirement data respectively, while the Python FastAPI ai-service is responsible for generating and refining document content. MongoDB serves as the persistence layer across all modules.

1.6 Literature Review

Traditional approaches to SDLC documentation rely on manually authored templates often maintained in word processors or wiki-style tools such as Confluence which are filled in by project managers, business analysts, and developers at each stage of a project. While flexible, this approach has well-documented drawbacks: documentation quality varies significantly between authors, terminology is inconsistently applied across a team, and keeping documents synchronized with an evolving project is a manual, easily neglected task.

Project management and issue-tracking tools such as Jira and Azure DevOps partially address this by structuring certain artifacts (backlogs, sprints, issues) but do not generate formal

documentation such as SRS, HLD/LLD, or test plans, and are themselves methodology-specific rather than adaptable across Waterfall, Agile, V-Model, and other models within a single platform.

More recently, the emergence of large language models (LLMs) has enabled a new class of tools capable of drafting structured technical documents from natural-language or structured input. However, general-purpose AI writing assistants are not SDLC-aware: they do not enforce methodology-specific terminology rules, do not understand the dependency between document types within a lifecycle (for example, that a Cost Estimation Sheet should draw on historical data from prior projects), and are prone to fabricating plausible-sounding but incorrect content when the necessary information is not actually available to them.

This project's approach differs by treating document generation as a structured, verifiable process rather than a single free-form generation step: content is produced against explicit per-model rules, verified in code rather than by re-prompting alone, and fixes are scoped to only the parts of a document that failed review. This positions the system between rigid, non-generative template tools on one end and unconstrained general-purpose AI writing tools on the other.

Capability	Confluence / Wiki Templates	Jira / Azure DevOps	Generic AI Writing Assistants	This Platform
Generates formal SDLC documents (SRS, HLD/LLD, test plans)	No free-text pages only	No structures backlog items, not formal docs	Can draft text, but not methodology-structured	Yes - 41 document types across 3 modules
Methodology-aware (Waterfall / Agile / V-Model, etc.)	No	Partially tool itself is methodology-specific	No - not SDLC-aware	Yes - 7 supported models with model-specific rules

Verifies fixes in code rather than trusting re-prompted text	N/A	N/A	No	Yes verification. py rule checks
Cross-project organizational knowledge base	No	No	No	Yes — Module B Risk Repository & Historical Data

Table 1.6: Comparison with Existing Tools

2 Project Management

Project management for this project centered on a feasibility study that justified undertaking it, the planning decisions that shaped how it was built, the roles and responsibilities involved, and the schedule the work followed.

2.1 Feasibility Study

Feasibility Type	Verdict
Technical (2.1.1)	Feasible — built on widely adopted, well-documented technologies (MERN stack, LLM APIs, RESTful microservices)
Time Schedule (2.1.2)	Feasible within the internship duration — divided into well-defined, independently testable development phases
Operational (2.1.3)	Feasible — simplifies a time-consuming manual activity via a simple wizard-based input flow
Implementation (2.1.4)	Feasible — all required frameworks, APIs, and development resources were available and successfully integrated

Table 2.1: Feasibility Study Summary

2.1.1 Technical Feasibility

The proposed AI-Assisted SDLC Documentation Generator is technically feasible because it is developed using widely adopted and well-supported technologies, frameworks, and development tools. The system is implemented using the MERN Stack, including React.js for the frontend, Node.js and Express.js for backend services, and MongoDB for storing project information and generated documentation. Artificial Intelligence capabilities are integrated through Large Language Models (LLMs) to automatically generate software engineering documents based on user-provided project requirements.

The application follows a microservices architecture, where different services are responsible for requirement processing, AI-based document generation, document management, UML diagram generation, and project lifecycle management. This modular design improves

maintainability, scalability, and independent deployment of services. RESTful APIs enable seamless communication between the frontend, backend services, and AI modules, ensuring efficient data exchange throughout the application.

Furthermore, the system utilizes Mermaid.js for automatic UML diagram generation and supports multiple Software Development Life Cycle (SDLC) methodologies, including Waterfall, Agile Scrum, Agile Kanban, Spiral, V-Model, RAD, and DevOps. Since all required technologies, software libraries, APIs, and development tools are readily available and well documented, the technical implementation of the proposed system is highly feasible.

The development environment requires only standard computing resources with internet connectivity, Visual Studio Code, Node.js, MongoDB, and access to AI services. Therefore, the project can be successfully implemented without requiring specialized hardware or expensive infrastructure.

2.1.2 Time Schedule Feasibility

The development of the AI-Assisted SDLC Documentation Generator is feasible within the allocated internship duration because the project has been divided into multiple well-defined development phases. The work includes requirement analysis, system architecture design, backend development, frontend implementation, AI prompt engineering, SDLC document generation, UML diagram generation, testing, debugging, and documentation.

The modular architecture adopted during development enabled different project components to be implemented independently while allowing smooth integration during later phases. This reduced development complexity and improved productivity throughout the internship.

Existing JavaScript frameworks, open-source libraries, AI APIs, and document generation utilities significantly reduced implementation time. Continuous testing after every completed module further minimized integration issues and ensured steady project progress.

Since the required technologies, development resources, and project requirements were available throughout the internship period, the complete implementation, testing, and validation of the proposed system could be accomplished within the planned project schedule.

2.1.3 Operational Feasibility

The proposed AI-Assisted SDLC Documentation Generator is operationally feasible because it simplifies one of the most time-consuming activities in software development, namely the preparation of software engineering documentation. Instead of manually creating dozens of documents, users only need to provide project requirements, select the preferred SDLC model, and specify project details.

Based on the provided inputs, the system automatically generates complete project documentation, including Software Requirement Specification (SRS), High-Level Design (HLD), Low-Level Design (LLD), Software Design Documents (SDD), Work Breakdown Structure (WBS), User Stories, Sprint Planning documents, API documentation, database schema, UML diagrams, test cases, deployment documentation, project reports, and various management documents.

The user interface is designed to be simple and intuitive so that software developers, students, project managers, and organizations can easily generate professional software documentation without requiring advanced technical knowledge. The generated documents are editable and downloadable, making the system practical for real-world software projects.

By reducing manual documentation effort, minimizing inconsistencies, and improving documentation quality, the system significantly enhances software development productivity while maintaining standardized documentation practices.

2.1.4 Implementation Feasibility

The implementation of the AI-Assisted SDLC Documentation Generator is entirely feasible because all required software technologies, frameworks, APIs, and development resources are readily available. The application is implemented using React.js, Node.js, Express.js, MongoDB, REST APIs, and modern JavaScript libraries, all of which are open-source and widely supported.

Artificial Intelligence functionality is integrated using Large Language Models to automate

document generation. Mermaid.js is used for generating UML diagrams dynamically, while MongoDB stores project information, generated documents, and lifecycle data efficiently.

The application follows a microservices architecture, enabling independent development and deployment of individual modules such as Requirement Management, AI Document Generation, UML Generation, Document Storage, Authentication, and Organizational Knowledge Management.

Throughout the internship, all required software tools, development environments, and supporting technologies were successfully integrated and tested. Functional validation of all supported SDLC models, document generation workflows, and UML diagram generation confirmed that the system operates reliably and satisfies the intended project objectives. Therefore, the implementation of the proposed system is considered technically achievable and practically feasible.

2.2 Project Planning

2.2.1 Development Approach & Justification

The development of the **AI-Assisted SDLC Documentation Generator** followed a systematic and iterative software development approach to ensure the successful implementation of all project modules. The overall project was divided into multiple phases, including requirement analysis, system architecture design, database design, backend development, frontend development, artificial intelligence integration, SDLC document generation, UML diagram generation, testing, validation, and deployment. Dividing the project into smaller development stages simplified the implementation process and enabled each module to be developed, tested, and integrated independently without affecting the overall system.

A microservices architecture was adopted to improve the modularity, scalability, and maintainability of the application. Individual services were developed for authentication, requirement management, AI-assisted document generation, UML diagram generation, document management, and organizational knowledge management. This architecture allows each service to operate independently while communicating efficiently through RESTful APIs, making future enhancements and maintenance significantly easier.

Artificial Intelligence was integrated into the application using Large Language Models (LLMs) to automate the generation of software engineering documents. Instead of manually preparing extensive project documentation, users provide project requirements once, and the system automatically generates structured documents based on the selected Software Development Life Cycle (SDLC) methodology. This approach minimizes manual effort, improves documentation consistency, and significantly reduces project documentation time.

The application was implemented using React.js for developing an interactive user interface, Node.js and Express.js for backend services, MongoDB for database management, and Mermaid.js for automatic UML diagram generation. These technologies were selected because of their flexibility, extensive community support, scalability, and compatibility with modern web application development. The chosen development approach ensured efficient implementation while maintaining high software quality and supporting future expansion of the system.

2.2.2 Milestones & Deliverables

The development of the AI-Assisted SDLC Documentation Generator was organized into multiple milestones to ensure systematic implementation of every project objective. Each milestone corresponds to a major development phase and contributes to the successful completion of the overall system. The completion of every milestone provided a specific deliverable that supported the subsequent phases of development and ensured continuous project progress.

Milestone	Deliverable
Requirement Analysis	Defined project objectives, functional requirements, non-functional requirements, and supported SDLC models
System Architecture Design	Designed microservices architecture, database schema, API flow, and overall system structure

Authentication & User Management	Secure user registration, login, authorization, and profile management
Requirement Management Module	Project requirement collection, storage, and requirement editing functionality
AI Document Generation Module	Automatic generation of SRS, HLD, LLD, SDD, WBS, API documentation, and other SDLC documents
UML Diagram Generation	Automatic generation of Use Case, Class, Sequence, Activity, Component, Deployment, and ER diagrams
SDLC Model Integration	Implementation of Waterfall, Agile Scrum, Agile Kanban, Spiral, V-Model, RAD, and DevOps document templates
Organizational Knowledge Repository	Storage of historical project information, reusable documentation, lessons learned, and project recommendations
Testing & Validation	Functional testing, document validation, UML verification, API testing, and bug fixing
Final Deployment & Documentation	Complete project deployment, user documentation, technical report preparation, and internship thesis submission

Table 2.2.2: Project Milestones and Deliverables

The successful completion of the above milestones resulted in the development of a comprehensive AI-Assisted SDLC Documentation Generator capable of generating complete software engineering documentation based on multiple SDLC methodologies. The final deliverable includes an integrated web-based application consisting of AI-powered document generation, UML diagram generation, project lifecycle management, organizational knowledge repository, and complete technical documentation prepared during the internship.

2.2.3 Roles & Responsibilities

During the internship, we were responsible for the design, development, implementation, testing, and documentation of the AI-Assisted SDLC Documentation Generator. our primary responsibility involved understanding project requirements, studying different Software Development Life Cycle methodologies, and designing an application capable of automatically generating standardized software engineering documents using Artificial Intelligence.

we participated in designing the system architecture, database schema, RESTful APIs, and frontend user interface. The backend services were implemented using Node.js and Express.js, while the frontend was developed using React.js. MongoDB was used for storing project information, generated documentation, and organizational knowledge. Additionally, AI prompt engineering was performed to generate accurate and structured SDLC documents corresponding to different software development models.

Another major responsibility included developing document generation templates for Waterfall, Agile Scrum, Agile Kanban, Spiral, V-Model, RAD, and DevOps methodologies. I also implemented automatic UML diagram generation using Mermaid.js and integrated all services into a unified application through REST APIs. Functional testing, debugging, prompt refinement, document validation, UML verification, performance improvement, and final project documentation were also carried out throughout the internship. These responsibilities provided practical experience in full-stack web development, software architecture, Artificial Intelligence integration, microservices, API development, and software engineering documentation.

2.2.4 Group Dependencies

The AI-Assisted SDLC Documentation Generator was developed through the integration of multiple interconnected modules, where the successful implementation of each component depended upon the completion of previous development stages. Requirement collection served as the foundation of the application because all generated documents, UML diagrams, and project lifecycle reports relied on the project information provided by the user.

The AI-powered document generation module depended upon the successful implementation of requirement management, prompt engineering, and document templates. Similarly, UML diagram generation relied on the generated software documentation and project metadata to automatically produce accurate software design diagrams. The frontend application depended upon backend services for authentication, project management, document generation, and data retrieval through RESTful APIs.

The Organizational Knowledge Repository relied on the successful completion of project documentation modules to store historical project information, reusable templates, project risks, lessons learned, and estimation data. Similarly, the project lifecycle management module depended upon all previously generated documents to produce execution reports, testing documentation, project completion reports, and maintenance records.

Finally, system integration, testing, validation, and deployment were carried out only after all individual modules were successfully implemented and verified. This dependency-driven development approach ensured smooth integration among all project components while improving the reliability, consistency, and overall quality of the application.

Component	Depends On
AI-Powered Document Generation	Requirement Management (project input)
UML Diagram Generation	Generated documentation & project metadata
Frontend Application	Backend services (auth, project management, document generation, data retrieval)
Organizational Knowledge Repository	Completed project documentation modules
Project Lifecycle Management	All previously generated documents
System Integration, Testing & Deployment	Successful, verified implementation of all individual modules

Table 2.2.4: Module Dependency Matrix

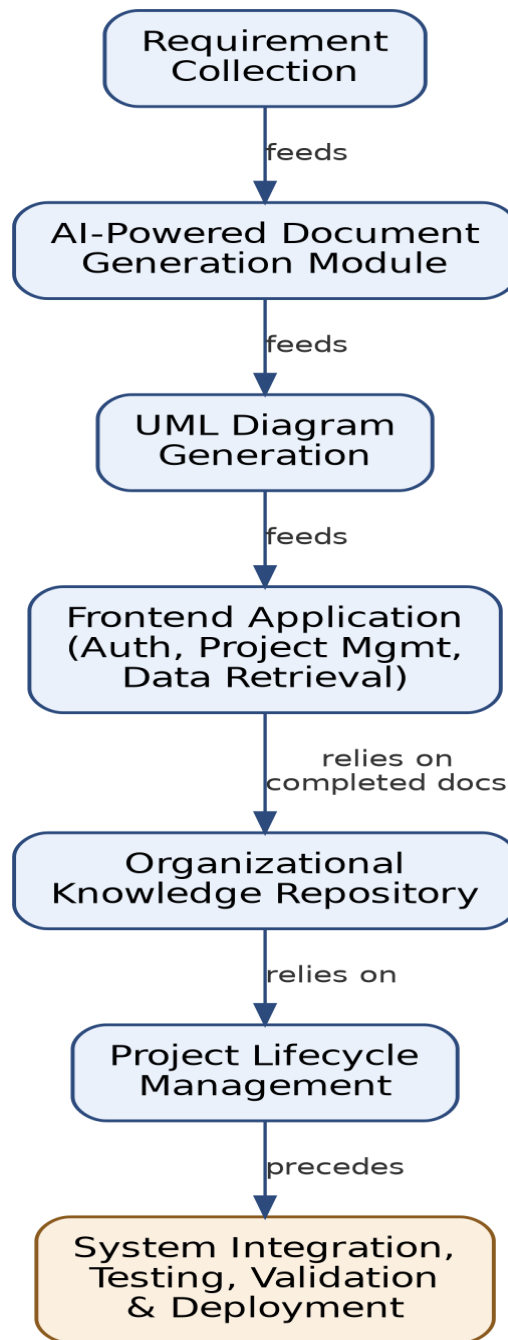


Figure 2.2.4: Module Dependency Diagram

2.3 Project Scheduling (Gantt/PERT Chart)

The Gantt Chart illustrates the planned schedule for the development of the AI-Assisted SDLC Documentation Generator over the thirteen-week internship period. It shows the sequence and duration of major activities such as requirement analysis, system development, AI integration, testing, and documentation.

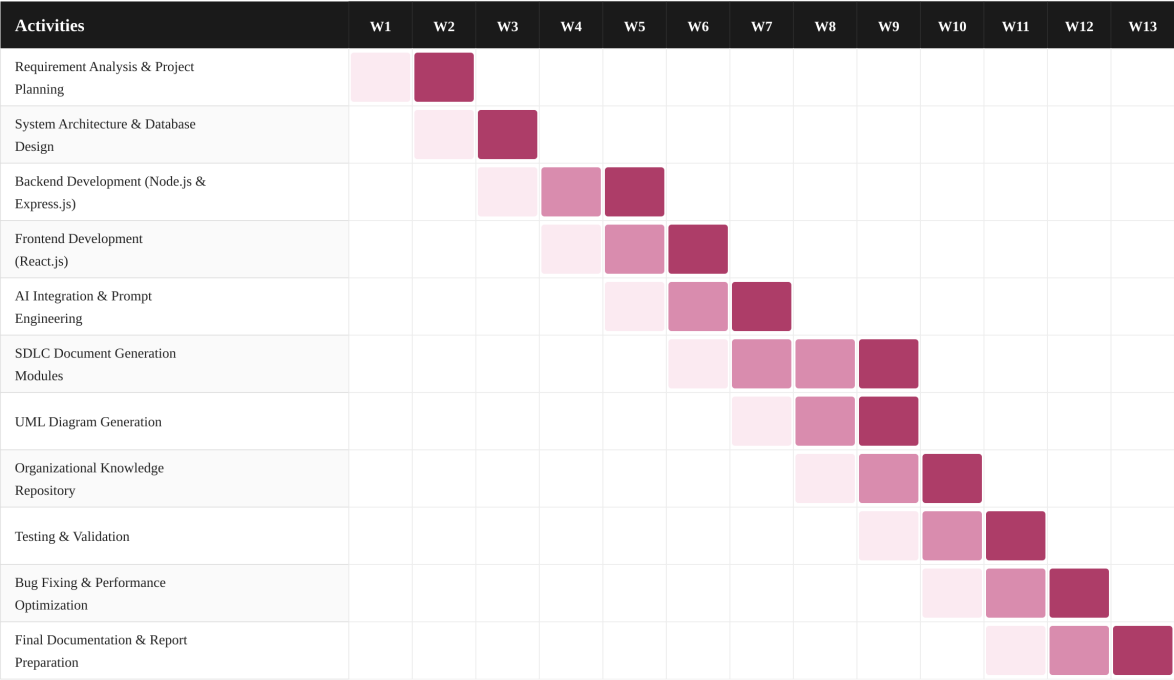


Figure 2.3: Gantt Chart of the Project Schedule

Figure 2.3: Gantt Chart of the Project Schedule

3 System Requirement Study

The requirement study that shaped this system's design examined the state of practice prior to the project, the limitations that motivated it, the intended users, and the functional, non-functional, hardware/software, and constraint-level requirements derived from that study.

3.1 Existing System Overview

Prior to this project, SDLC documentation within the organization was produced using a combination of manually authored templates (word processor documents) and general-purpose collaboration tools. Project managers, business analysts, and developers were individually responsible for drafting requirement, design, planning, and test documents at each stage of a project, typically starting from a static template that was not specific to the SDLC methodology in use. Review and correction of these documents was a manual process, issues were identified by a reviewer reading the full document and communicating changes back to the author, who would then edit the document directly.

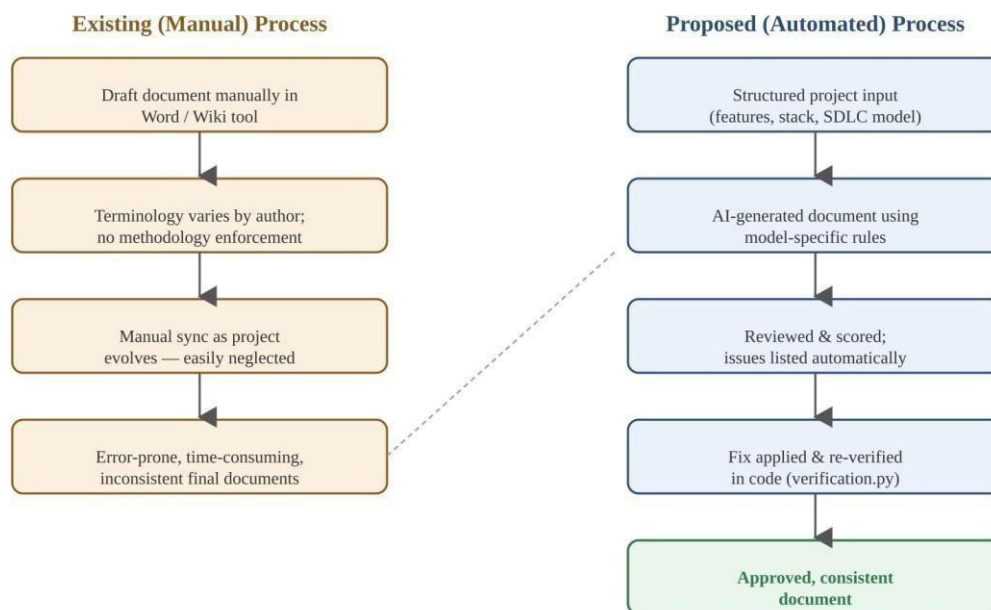


Figure 3.1: Existing (Manual) Process vs. Proposed (Automated) Process

Figure 3.1: Existing (Manual) Process vs. Proposed (Automated) Process

3.2 Limitations of the Existing System

- Documentation quality and completeness varied significantly depending on the individual author, since templates provided structure but not content guidance.
- Terminology and structure were not enforced against the project's SDLC methodology, so it was possible (and common) for a Waterfall project's documents to contain Agile terminology, or vice versa, without this being caught until review.
- There was no systematic reuse of risk history or cost/effort estimates from past projects, each new project's estimation typically started from zero, relying on individual memory rather than an organizational record.
- Recurring execution-phase artifacts (meeting minutes, issue logs, status reports) were tracked in disparate places (spreadsheets, email, shared drives) with no single dashboard showing what was due or outstanding.
- Fixing an issue found during review meant manually locating and editing the relevant section of a document, with no automated check confirming the fix actually resolved the flagged issue.
- Workforce training records (skills, training needs, sessions delivered, evaluations) were maintained separately from project documentation, with no automatic link between identified skill gaps and recommended training programs.

Limitation	Impact
Documentation quality varied by author	Templates gave structure but not content guidance, so quality was inconsistent
No enforcement of SDLC terminology/structure	A Waterfall project's documents could contain Agile terminology (or vice versa) undetected until review
No systematic reuse of risk/estimation history	Every new project's estimation started from zero, relying on individual memory
Execution-phase artifacts tracked in disparate places	No single dashboard for meeting minutes, issue logs, or status reports

No automated check that a fix resolved an issue	Fixing required manually locating and editing a section, with no verification step
Training records maintained separately from projects	No automatic link between identified skill gaps and recommended training

Table 3.2: Limitations of the Existing System

3.3 User Characteristics

User Type	Characteristics
Project Contributor / Developer	Uses Module A day-to-day to generate and refine project documents; may have little to no prior formal SDLC documentation training
Project Manager	Oversees Stage 0 approval, Stage 2 planning documents, and the Stage 3 execution dashboard; needs a consolidated view of document status
Organizational Administrator	Configures SDLC model defaults, manages people records shared across Modules A and C, and reviews Module B's accumulated knowledge base
Trainer / Training Coordinator	Uses Module C to run skill assessments, log training sessions, and review evaluation and outcome reports

Table 3.3: User Characteristics

Across all user types, the system was designed on the assumption that users are comfortable with standard web application interfaces but should not be assumed to have specialist SDLC documentation expertise, this assumption directly shaped the objective (Section 1.4) of requiring no prior SDLC experience to use the system.

3.4 Functional Requirements

- The system shall allow a user to create a project by specifying its name, description, features, modules, technology stack, team roles, and constraints.
- The system shall require selection of one of seven supported SDLC models (Waterfall, Agile-Scrum, Agile-Kanban, V-Model, Spiral, RAD, DevOps) at project creation.
- The system shall gate access to the Stage 1 document generation wizard behind approval of a Stage 0 Opportunity Sheet.
- The system shall generate each document type using rules specific to the project's selected SDLC model, including terminology restrictions and feature-grouping structure.
- The system shall score each generated document and list individual issues found during review.
- The system shall accept a user-described or suggested fix, apply it to only the affected part of a document, and re-verify the result in code before accepting it.
- The system shall maintain an automatically updated Master Data List reflecting every document generated for a project, its version, and its status.
- The system shall group Stage 3 execution documents by recurrence type (log, stateful log, evolving) on a dedicated execution dashboard.
- The system shall surface Module B risk and historical estimation suggestions to a project without applying them automatically, requiring explicit accept/edit/reject action.
- The system shall support Module C workflows independently of Module A, including skill gap identification, training need prioritization, session logging, and evaluation reporting.
- The system shall allow any generated document to be exported as a PDF from its review screen.

Req. ID	Requirement
FR-01	Allow a user to create a project by specifying name, description, features, modules, tech stack, team roles, and constraints
FR-02	Require selection of one of seven supported SDLC models at project creation
FR-03	Gate access to the Stage 1 generation wizard behind Stage 0 Opportunity Sheet approval
FR-04	Generate each document type using rules specific to the project's selected SDLC model
FR-05	Score each generated document and list individual issues found during review
FR-06	Accept a user-described or suggested fix, apply it to only the affected part, and re-verify in code
FR-07	Maintain an automatically updated Master Data List reflecting every document, its version, and status
FR-08	Group Stage 3 execution documents by recurrence type on a dedicated dashboard
FR-09	Surface Module B risk/historical suggestions without applying them automatically
FR-10	Support Module C workflows independently of Module A
FR-11	Allow any generated document to be exported as a PDF from its review screen

Table 3.4: Functional Requirements of the Proposed System

3.5 Non-Functional Requirements

Category	Requirement
Usability	A first-time user should be able to complete a project's core documentation without prior formal SDLC training
Consistency	Terminology and structure rules must be enforced identically for a given SDLC model across all document types
Reliability	A fix confirmed as correct for one SDLC model must not be assumed correct for another without independent verification

Data integrity	The system must never fabricate content for fields it has not been given data for; such fields must be presented as blank templates
Maintainability	Each backend service (api-gateway, document-service, requirement-service, ai-service) should be independently deployable and testable
Performance	Target document generation time is under 30 seconds per document via the AI service, with verification and review checks completing in under 5 seconds.
Scalability	The system should support at least 50 concurrent active users and manage over 1,000 discrete project records in its initial Docker-based deployment, scalable horizontally if migrated to container orchestration (e.g., Kubernetes).

Table 3.5: Non-Functional Requirements of the Proposed System

3.6 Hardware & Software Requirements

Category	Requirement
Frontend framework	React
Backend runtime	Node.js (api-gateway, document-service, requirement-service)
AI service runtime	Python 3.x, FastAPI
Database	MongoDB
Development hardware	Multi-core processor (Intel i5 / Apple M1 equivalent or better), minimum 8GB RAM (16GB recommended to comfortably run all Docker microservices locally), and at least 20GB of free disk space.
Server / deployment hardware	Cloud-based Virtual Private Server (VPS) or on-premise host with a minimum of 4 vCPUs, 16GB RAM, and 50GB SSD storage, capable of running Docker daemon and Docker Compose.
Browser support	Modern evergreen web browsers: Google Chrome (v90+), Mozilla Firefox (v88+), Apple Safari (v14+), and Microsoft Edge (v90+).

Table 3.6: Hardware & Software Requirements

3.7 Constraints

Constraint Type	Summary
UI (3.7.1)	Interface adapts fields dynamically based on the selected SDLC model
Communication Interface (3.7.2)	All services communicate over REST/HTTP through the api-gateway
Hardware Interface (3.7.3)	No specialized hardware required beyond standard development/deployment machines

Criticality of Application (3.7.4)	Draft-quality output — not a substitute for stakeholder review or compliance sign-off
Safety & Security (3.7.5)	Access control and secure transport (HTTPS/TLS) required for organization-internal deployment

Table 3.7: Constraints Summary

3.7.1 UI Constraints

The interface must present the correct wizard fields for the selected SDLC model, four of the seven models (Scrum, Spiral, V-Model, RAD) require additional fields (e.g. Sprint Length, Known Risk Areas, Criticality Level, Timebox Duration) beyond the base project form, and the UI must adapt to show only the fields relevant to the chosen model.

3.7.2 Communication Interface

The frontend communicates with backend services through the api-gateway, which routes requests to document-service, requirement-service, or the ai-service as appropriate. RESTful HTTP APIs are used for all inter-service and client-server communication. The API Gateway routes requests and enforces security using JSON Web Tokens (JWT) passed in the Authorization: Bearer header. Internal microservices communicate seamlessly over a private Docker bridge network (sdhc-network).

3.7.3 Hardware Interface

The system has no dependency on specialized hardware; it is a web-based platform accessed through a standard browser over a network connection to the deployed services. The system runs on standard commercial server hardware. If deployed on-premise at BISAG-N, it will be accessed via the organization's internal LAN/Intranet, potentially requiring secure VPN access for remote developers depending on internal IT network policies.

3.7.4 Criticality of Application

The system produces draft, review-ready documentation rather than final, unreviewed deliverables, it is not classified as safety-critical. However, correctness of the fix-verification logic (verification.py) is operationally important, since an incorrectly "passed" document could propagate an unresolved issue into downstream stages (e.g. an unverified Stage 1 document

feeding into Stage 2 planning).

3.7.5 Safety & Security Considerations

- Project data (Module A) and training/personnel data (Module C) must not be conflated, since they use separate entity models.
- Suggestions drawn from Module B's organizational knowledge base must never be applied without explicit user action, to prevent one project's data silently altering another's documents.
- Authentication and session management are handled via JSON Web Tokens (JWT) through a dedicated Auth Service. All passwords are one-way hashed using bcrypt. Production deployments must secure data-in-transit by placing the API Gateway behind a reverse proxy (e.g., Nginx) configured with HTTPS/TLS 1.2+. Access control will isolate project data to prevent unauthorized cross-project modifications.

3.8 Assumptions & Dependencies

- It is assumed that a project's SDLC model, once chosen at creation, will not need to change, the system does not currently support migrating a project between models.
- It is assumed that documents needing real-world input (e.g. actual test results) will be supplied by the user; the system does not attempt to infer this data.
- Module B's usefulness depends on prior projects having been completed within the system, it has no access to project history that predates the system or was never entered.
- The system depends on the availability and correctness of the AI generation service (ai-service) for first-draft content; verification.py governs correctness of accepted fixes but the initial draft's quality still depends on this service.
- The system critically depends on external third-party LLM API providers (OpenAI, Google Gemini, Groq) for first-draft document generation. It is assumed these APIs will remain highly available. The speed and volume of document generation are directly constrained by the rate limits and token quotas established by the active API subscription tiers on these platforms.

Type	Description
Assumption	Users are comfortable with standard web application interfaces
Assumption	Users are not assumed to have specialist SDLC documentation expertise
Dependency	AI generation quality depends on the availability/performance of external LLM providers (Gemini, Groq)
Dependency	MongoDB availability is required across all four services for any functionality to work

Table 3.8: Assumptions & Dependencies

4 Proposed System Requirements

4.1 Overview of Proposed System

The proposed system is a web-based platform that generates SDLC documentation directly from a structured description of a software project, and refines that documentation through an iterative, code-verified review cycle rather than one-off generation. A project is described once its name, description, features, modules, technology stack, team roles, constraints, and chosen SDLC model and the system produces a complete, methodology-correct set of documents from that description, updating them on request rather than requiring manual rewriting.

The system extends beyond single-project documentation in two directions: an organizational knowledge base that lets new projects draw on risk and estimation history from past projects, and a separate organizational training subsystem that manages workforce skill assessment and training delivery. Together these form a platform organized into three modules Project Lifecycle, Organizational Knowledge Base, and Organizational Training described in Section 4.2.

4.2 Module Descriptions

4.2.1 Module A Project Lifecycle

Covers a single project from initial approval through closure across five stages: Stage 0 (Opportunity an approval gate before full document generation unlocks), Stage 1 (SDLC Generation — the 12 core technical documents), Stage 2 (Planning Refinement cost, budget, and requirement-validation documents, plus an auto-maintained Master Data List), Stage 3 (Execution recurring documents grouped by how often they repeat: log, stateful log, or evolving), and Stage 4 (Closure a completion/learning report that feeds Module B).

4.2.2 Module B Organizational Knowledge Base

A cross-project store of two kinds of shared knowledge: a Risk Repository of risks encountered on past projects and what mitigation worked, and Historical Data from previous projects' cost/effort estimates versus actuals. Both are surfaced as optional suggestions to new projects with similar tech stack, constraints, or SDLC model — never applied automatically.

4.2.3 Module C Organizational Training

A workforce-training subsystem, functionally independent of Modules A and B, with its own entity model for employees, trainers, training programs, and skills. It identifies skill gaps, recommends training programs, logs delivered sessions, and produces evaluation and outcome reports including an auto-derived priority ranking for training needs and auto-suggested training programs where a skill match exists.

Module	Purpose	Key Functions
Module A — Project Lifecycle	Manage a single project from approval through closure	Stage 0 approval gate; Stage 1 SDLC generation; Stage 2 planning refinement; Stage 3 execution logging; Stage 4 closure
Module B — Organizational Knowledge Base	Cross-project reusable knowledge	Risk Repository; Historical cost/effort data; accept/edit/reject suggestion workflow
Module C — Organizational Training	Workforce training, independent of Modules A/B	Skill gap identification; training program recommendation; session logging; evaluation & outcome reporting

Table 4.2: Modules & Functions of the Proposed System

4.3 System Features

- Methodology-aware generation: documents are produced against rules specific to the project's chosen SDLC model, covering terminology, structure, and feature-grouping (by phase, sprint, board column, V&V pairing, risk-ordered iteration, timebox, or pipeline stage, depending on the model).
- Generate-review-fix cycle: every document is scored on generation, issues are listed individually, and a fix — whether suggested or user-described — is verified in code before being accepted, rather than trusting a re-prompted result.
- No fabricated content: fields requiring real-world input the system does not have are presented as blank templates rather than guessed values.
- Auto-maintained Master Data List: a continuously updated register of every document generated for a project, its version, and its status, requiring no manual upkeep.

- Execution dashboard: Stage 3 documents grouped by recurrence type (log, stateful log, evolving) so a project's day-to-day documentation state is visible in one place.
- Cross-project knowledge reuse: Module B suggestions let new projects draw on organizational risk and estimation history instead of starting from zero, with the user always able to accept, edit, or reject.
- Independent training subsystem: Module C automates skill-gap identification, training-need prioritization, and evaluation reporting without depending on SDLC-model logic.
- PDF export: any generated document can be exported for sharing outside the platform.

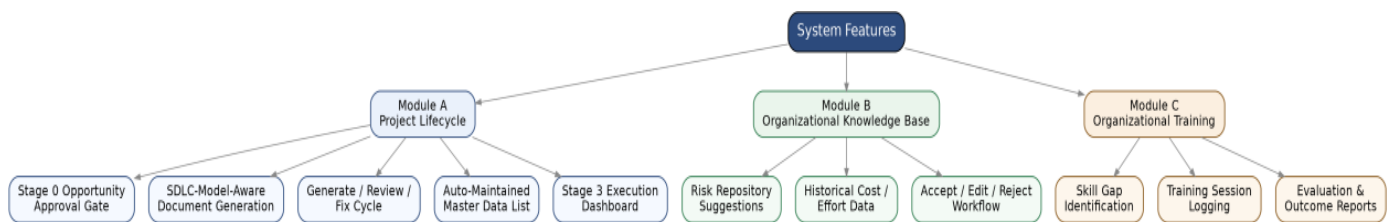


Figure 4.3: System Features Map

4.4 Advantages of Proposed System

Measured against the limitations identified in Section 3.2, the proposed system offers the following advantages:

- Consistent documentation quality: automated methodology rules replace author-dependent template usage, so a document's terminology and structure no longer depend on which individual drafted it.
- Enforced methodology correctness: terminology and structure are checked per SDLC model, closing the gap that previously allowed mismatched vocabulary to reach review undetected.
- Reusable organizational knowledge: risk and estimation history is captured once and made available to every future project, rather than relying on individual memory.
- A single execution view: recurring documentation that was previously scattered across spreadsheets, email, and shared drives is consolidated into one dashboard, grouped by how it behaves.
- Verifiable fixes: because fixes are checked in code against explicit rules, a document marked as resolved has actually been confirmed against those rules, rather than assumed correct after a rewrite.
- Linked training management: skill gaps and training recommendations are connected automatically, rather than maintained as separate, unlinked records.

Table 4.4: Advantages of the Proposed System

Advantage	Resolves Limitation
Consistent documentation quality via automated methodology rules	Author-dependent template usage (Table 3.2, row 1)
Enforced methodology correctness per SDLC model	No terminology/structure enforcement (row 2)
Reusable organizational knowledge (risk & estimation history)	No systematic reuse of history (row 3)
Single consolidated execution dashboard	Artifacts scattered across tools (row 4)
Verifiable fixes checked in code	No automated fix verification (row 5)
Linked skill gaps and training recommendations	Disconnected training records (row 6)

5 System Design

5.1 System Architecture Design

The system follows a microservice architecture rather than a single monolithic application. A React frontend communicates with an api-gateway, which routes requests to three backend services: document-service and requirement-service (Node.js), and ai-service (Python, FastAPI), which performs document generation and hosts the code-level verification gates (verification.py) that check fixes before they are accepted. All services share a single MongoDB instance for persistence. This separation allows each service to be built, tested, and deployed independently, which directly supported the phased development approach described in Section 2.2.1.

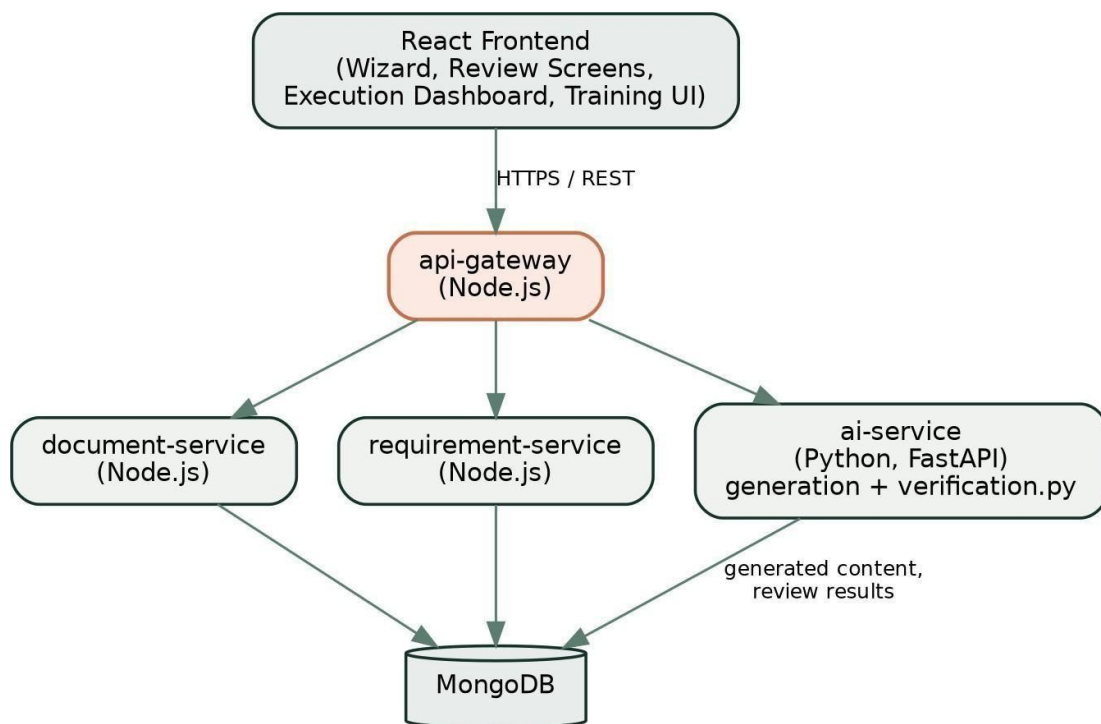


Figure 5.1: System Architecture Diagram

5.2 UML Diagrams

The following diagrams model the system from complementary perspectives: static structure (E-R and Class diagrams), user interaction (Use Case diagram), runtime behavior (Sequence and Activity diagrams), data movement (DFD), and physical deployment (Deployment diagram).

Diagram	What It Shows	Section
E-R Diagram	Entity relationships across MongoDB collections	5.2.1
Use Case Diagram	User-type interactions with the system	5.2.2
Class Diagram	Structural relationships between core entities	5.2.3
Sequence Diagram	Generate/Review/Fix cycle message flow	5.2.4
Activity Diagram	Project approval to document pass workflow	5.2.5
DFD (Level 0 & 1)	Data flow through the system at increasing detail	5.2.6
Deployment Diagram	Physical/infrastructure placement of services	5.2.7
Component Diagram	Service-level components and their interfaces	5.2.8

Table 5.2: UML Diagram Types & Purpose

5.2.1 E-R Diagram

The entity-relationship model centers on Project, from which Documents, Master Data List entries, and Execution Documents are derived. Module B's Risk Repository and Historical Estimate entities draw from completed projects but are not owned by any single project. Module C's Employee, Training Program, Training Session, and Evaluation entities form an independent cluster with no foreign-key relationship into Module A.

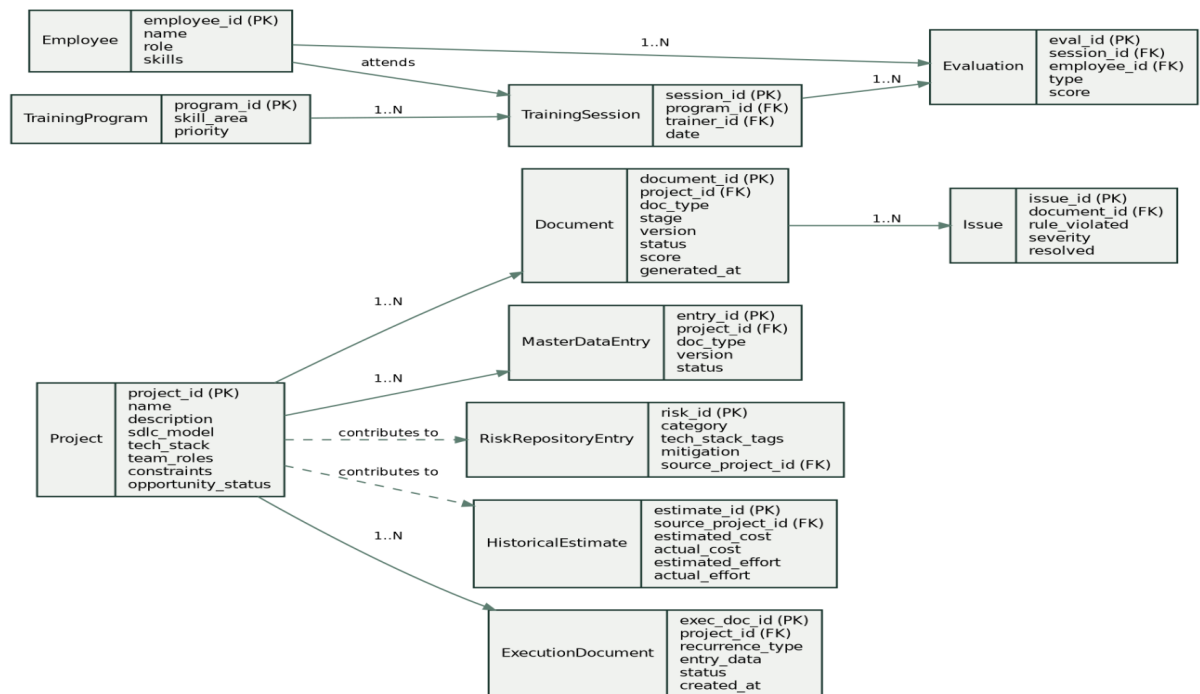


Figure 5.2.1: Entity-Relationship Diagram

5.2.2 Use Case Diagram

Four actors interact with the system, each scoped to the responsibilities described in Section 3.3: the Project Contributor drives day-to-day document generation and fixes; the Project Manager owns approvals and the execution dashboard; the Organizational Administrator configures shared settings; and the Trainer/Coordinator operates Module C independently of the other two roles.

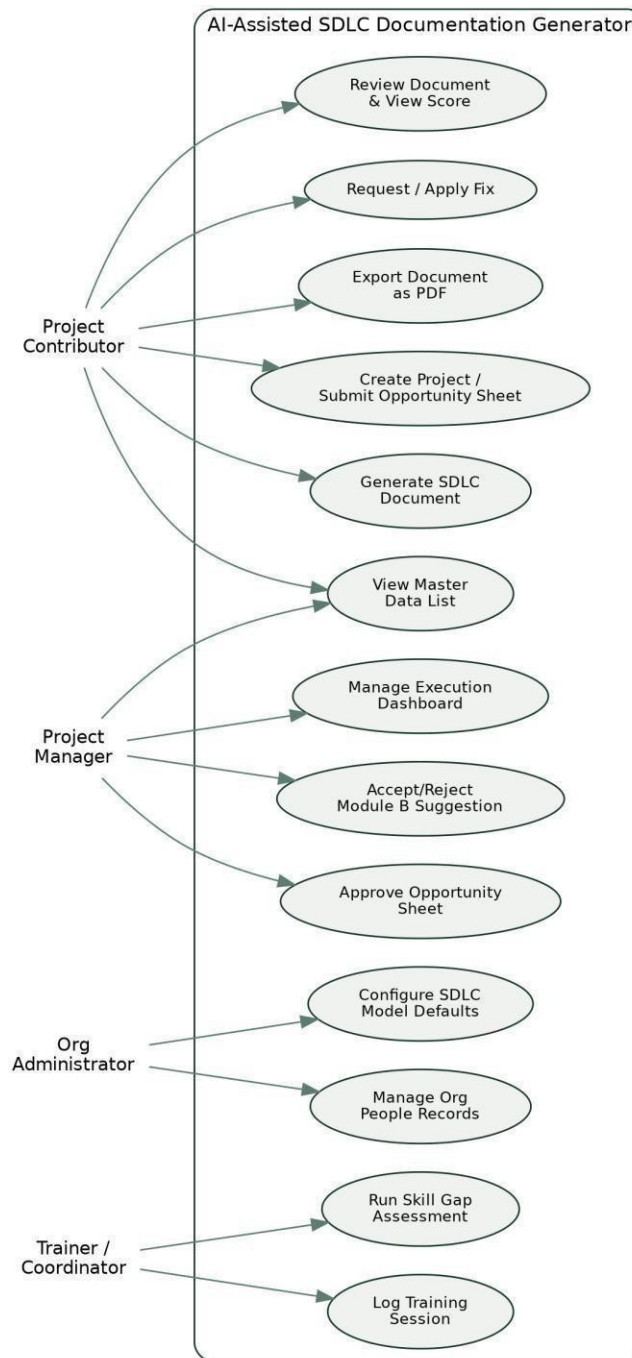


Figure. 5.2.2: Use Case Diagram

5.2.3 Class Diagram

Document composition and verification are the core relationships: a Project owns many Documents, each Document owns the DocumentIssues raised against it, and every fix is checked by a shared VerificationGate before being accepted — this is the class-level expression of the code-verified fix rule described in Section 1.6 and Section 4.3.

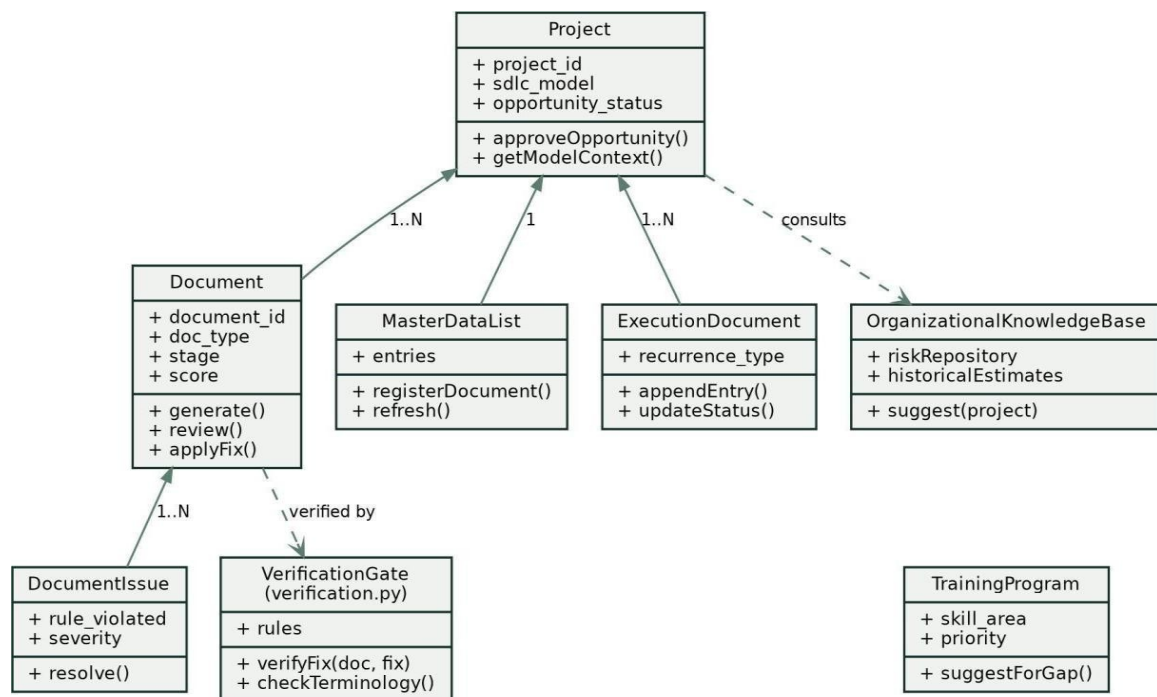


Figure 5.2.3: Class Diagram

5.2.4 Sequence Diagram

The sequence below traces a single generate/review/fix cycle (Section 3 of the user-facing workflow) across the frontend and backend services, showing where verification.py sits in the flow between a fix being proposed by ai-service and a revised score being returned, rather than as a separate offline step.

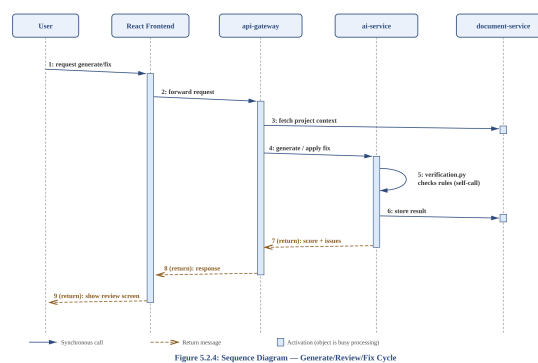


Figure 5.2.4: Sequence Diagram — Generate/Review/Fix Cycle

5.2.5 Activity Diagram

This diagram models the same cycle as a control-flow graph, including the Stage 0 approval gate that must be satisfied before generation begins, and the loop between review and fix that continues until a document passes.

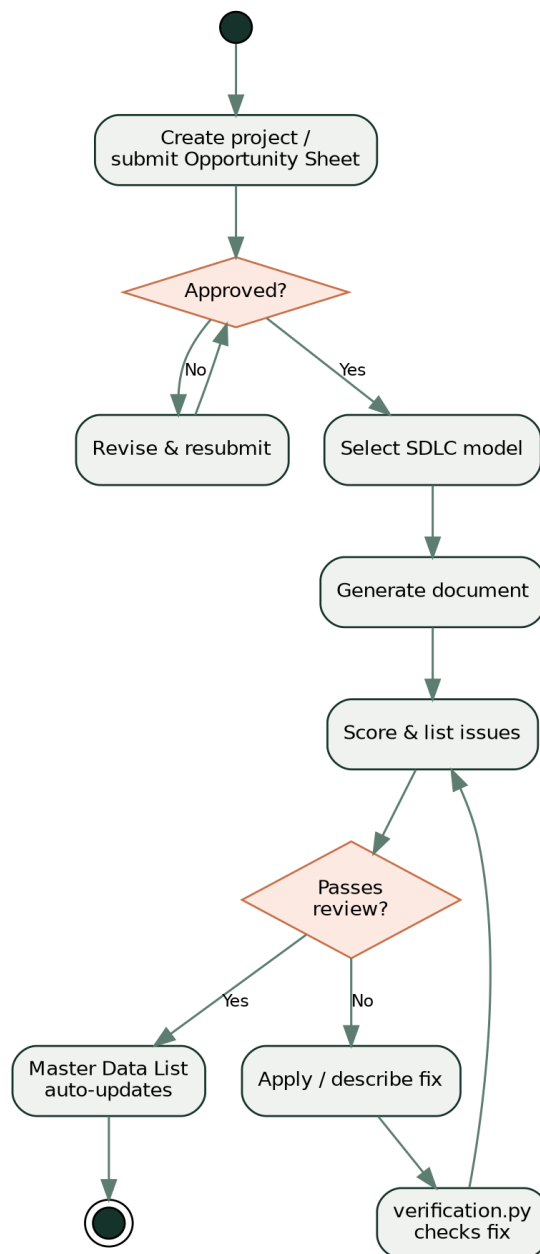


Figure 5.2.5: Activity Diagram — Project Approval to Document Pass

5.2.6 DFD Diagram

The Level 0 (context) diagram treats the entire platform as a single process exchanging project inputs and generated documents with the user. The Level 1 diagram decomposes this into five processes — project intake, document generation (which also draws on Module B's knowledge base), review and scoring, fix application, and Master Data List maintenance.

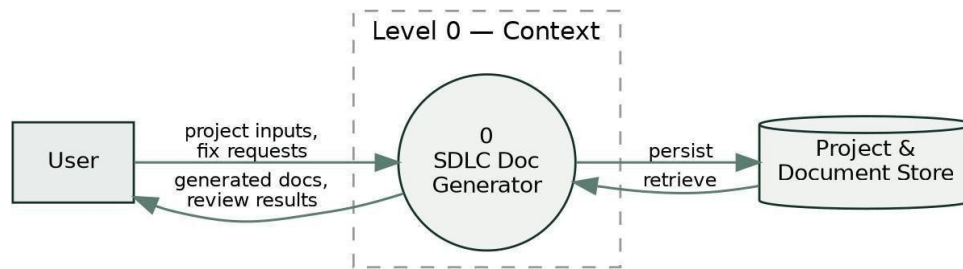


Figure 5.2.6 (i): Data Flow Diagram — Level 0 (Context)

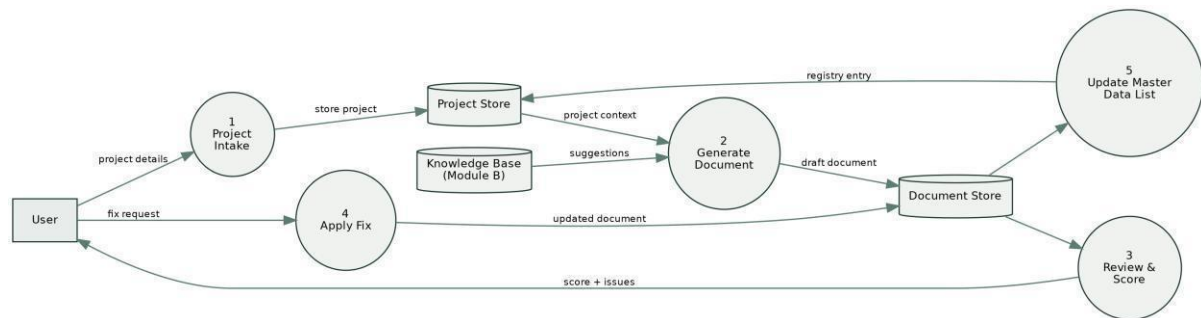


Figure 5.2.6 (ii): Data Flow Diagram — Level 1

5.2.7 Deployment Diagram

The deployed system spans three tiers: a client tier (the browser running the React single-page application), an application-server tier hosting the four backend services on BISAG-N infrastructure, and a data tier running MongoDB. All backend services communicate with MongoDB directly rather than through document-service alone, since requirement-service and ai-service each own distinct collections.

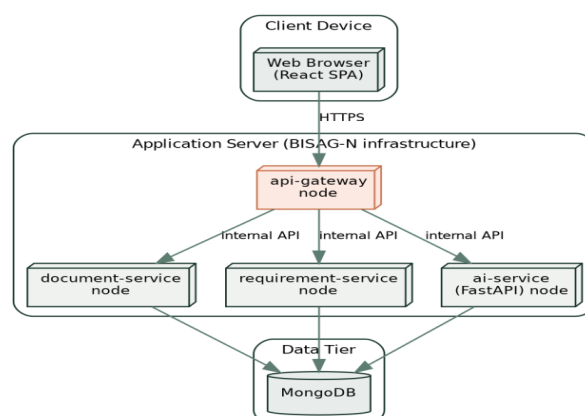


Figure 5.2.7: Deployment Diagram

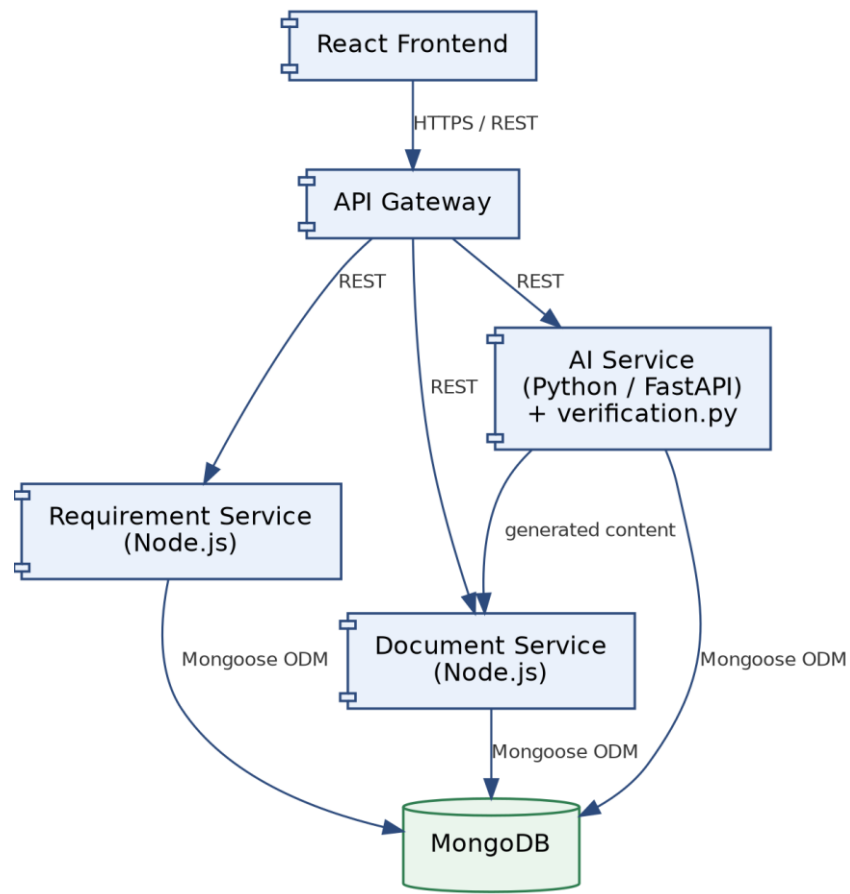


Figure 5.2.8: Component Diagram

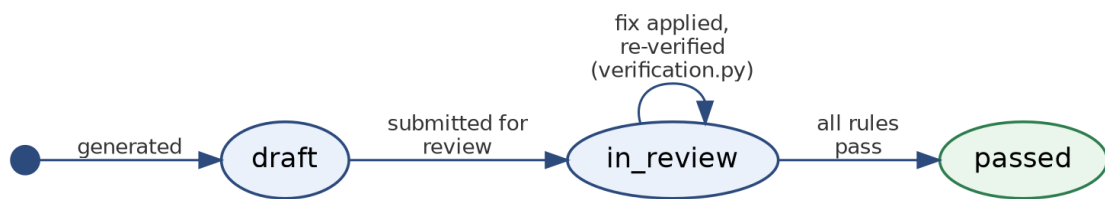


Figure 5.2.9: Document Status State Diagram

5.3 Database Design

5.3.1 Table Design & Relationships

MongoDB collections are used rather than relational tables, but the logical relationships mirror those in the E-R diagram (Section 5.2.1). The primary collections are:

Collection	Key Fields	Related To
projects	project_id, sdlc_model, opportunity_status	documents, master_data_entries, execution_documents
documents	document_id, project_id, doc_type, stage, score	issues
issues	issue_id, document_id, rule_violated, severity	documents
master_data_entries	entry_id, project_id, doc_type, version	projects
execution_documents	exec_doc_id, project_id, recurrence_type	projects
risk_repository	risk_id, category, tech_stack_tags, source_project_id	projects (reference only)
historical_estimates	estimate_id, source_project_id, estimated/actual cost & effort	projects (reference only)
employees	employee_id, name, skills	training_sessions, evaluations
training_programs	program_id, skill_area, priority	training_sessions
training_sessions	session_id, program_id, trainer_id, date	evaluations

Table 5.3.1: MongoDB Collections & Relationships

5.3.2 Normalization

As a document-oriented (MongoDB) rather than relational database, traditional normal-form normalization does not directly apply; the design instead follows document-database modeling conventions. Data that is always read together (e.g. a Document and its list of Issues) is considered for embedding rather than referencing, while data that is shared across many parents (e.g. Risk Repository entries referenced by multiple projects) is kept in its own collection and referenced by ID to avoid duplication.

Collection	Design Decision	Reason
projects	Embeds features and tech_stack as arrays rather than separate collections	These are always read together with the project and never queried independently
documents	References project_id (FK) rather than embedding the full project	Avoids duplicating project data across many documents

execution_documents	Embeds entry_data as a nested object	Entry structure varies by recurrence type, so a flexible embedded shape avoids a rigid schema
---------------------	--------------------------------------	---

Table 5.3.2: Denormalization / Embedding Decisions

The design uses a hybrid document-modeling approach: strictly bound metadata (such as a project's opportunity_sheet, features, and tech_stack arrays) is embedded directly within the projects collection to optimize read performance and minimize joins. Conversely, unbounded or independently updated entities (such as documents, issues, and master_data_entries) are stored in separate collections and referenced via foreign keys (e.g., project_id, document_id). This deliberate denormalization avoids unbounded array growth and ensures the system respects MongoDB's 16MB document size limit.

5.3.3 Data Dictionary

A representative excerpt of the data dictionary for the documents collection is given below; the complete dictionary should list every field across all collections in Section 5.3.1.

Table 5.3.3: Data Dictionary

documents

Field	Type	Description
document_id	ObjectId	Primary key, auto-generated
project_id	ObjectId (FK)	References the owning project
doc_type	String	One of the 41 supported document types
stage	Integer (0-4) / null	Module A stage; null for Module B/C documents
score	Integer (0-100)	Most recent review score
status	String	e.g. draft, in_review, passed
generated_at	ISODate	Timestamp of most recent generation

projects

Field	Type	Description
project_id	ObjectId	Primary key, auto-generated
title	String	Project name
sdic_model	String	Chosen model (e.g., 'Agile-Scrum', 'Waterfall')
opportunity_status	String	e.g., 'Under Review', 'Approved'
features	Array of Strings	Embedded array of key system features
tech_stack	Array of Strings	Embedded array of technologies used

issues

Field	Type	Description
issue_id	ObjectId	Primary key, auto-generated
document_id	ObjectId (FK)	References the document this issue belongs to
rule_violated	String	The specific methodology rule broken
severity	String	e.g., 'Warning', 'Critical'
resolved	Boolean	True if verified and fixed

master_data_entries

Field	Type	Description
entry_id	ObjectId	Primary key, auto-generated
project_id	ObjectId (FK)	References the owning project
doc_type	String	Document classification
version	Integer	Auto-incrementing version number
status	String	Current lifecycle state

execution_documents

Field	Type	Description
exec_doc_id	ObjectId	Primary key, auto-generated
project_id	ObjectId (FK)	References the owning project
recurrence_type	String	e.g., 'log', 'stateful_log', 'evolving'
entry_data	Object	Embedded execution details
created_at	ISODate	Timestamp of entry creation

risk_repository

Field	Type	Description
risk_id	ObjectId	Primary key, auto-generated
category	String	Risk classification (e.g., Technical, Schedule)
tech_stack_tags	Array of Strings	Associated technologies for matching
source_project_id	ObjectId (FK)	Reference to project where risk occurred
mitigation	String	Successful mitigation strategy applied

historical_estimates

Field	Type	Description
estimate_id	ObjectId	Primary key, auto-generated
source_project_id	ObjectId (FK)	Reference to the source project
estimated_cost	Number	Original budget estimation
Field	Type	Description
actual_cost	Number	Final recorded cost
estimated_effort	Number	Original time/sprint estimation
actual_effort	Number	Final recorded effort

employees

Field	Type	Description
employee_id	ObjectId	Primary key, auto-generated
name	String	Full name of employee
role	String	Job title or primary role
skills	Array of Strings	Assessed skill proficiencies

training_programs

Field	Type	Description
program_id	ObjectId	Primary key, auto-generated
skill_area	String	Domain or technology covered
priority	Integer	Auto-derived need ranking

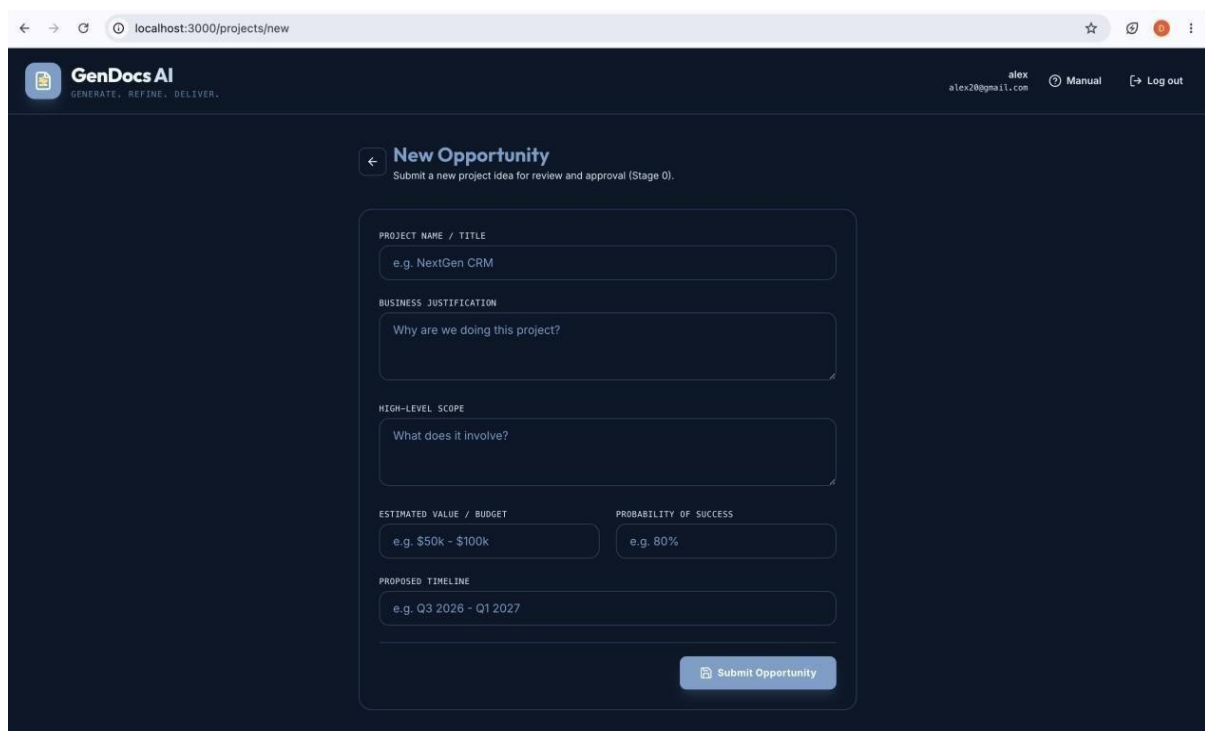
training_sessions

Field	Type	Description
session_id	ObjectId	Primary key, auto-generated
program_id	ObjectId (FK)	References the training program
trainer_id	ObjectId (FK)	References the employee conducting training
date	ISODate	Date of session delivery

5.4 GUI Design

The interface is organized around the same structure as the underlying system: a project wizard that adapts its fields to the selected SDLC model (Section 3.7.1), per-document review screens showing score and itemized issues, a Stage 3 execution dashboard grouping documents by recurrence type, and a separate Module C interface for training management. Navigation is kept flat — no more than two levels deep from the main dashboard to any document's review screen — to keep the generate-review-fix loop (Section 3) fast to repeat.

As the system has progressed past the initial prototyping phase into a fully implemented React frontend, low-fidelity wireframes have been superseded by the final user interface. Please refer to Section 5.5 for high-fidelity screenshots of the production UI, including the dynamic project wizard and the split-pane review screens.



The screenshot shows a web browser window with the URL `localhost:3000/projects/new`. The page features a dark blue header with the GenDocs AI logo and tagline 'GENERATE. REFINE. DELIVER.' on the left, and user information 'alex alex2@gmail.com', a 'Manual' link, and a 'Log out' button on the right. The main content area is titled 'New Opportunity' with a subtitle 'Submit a new project idea for review and approval (Stage 0)'. Below this is a form with several input fields: 'PROJECT NAME / TITLE' (with example 'e.g. NextGen CRM'), 'BUSINESS JUSTIFICATION' (with prompt 'Why are we doing this project?'), 'HIGH-LEVEL SCOPE' (with prompt 'What does it involve?'), 'ESTIMATED VALUE / BUDGET' (with example 'e.g. \$50k - \$100k'), 'PROBABILITY OF SUCCESS' (with example 'e.g. 80%'), and 'PROPOSED TIMELINE' (with example 'e.g. Q3 2026 - Q1 2027'). A 'Submit Opportunity' button is located at the bottom right of the form.

Screenshot 5.4.(i): Stage 0 — Opportunity Sheet Approval Screen (Wireframe)

GenDocs AI
GENERATE. REFINE. DELIVER.

alex

alex28@gmail.com

Manual

Log out

MODULE B: ORGANIZATION KNOWLEDGE BASE

Cross-Project Risk Repository & Historical Data Store

Global registry querying across all projects in the organization. Explore historical risk mitigations and estimation benchmarks categorized by SDLC model, technology stack, and engineering constraints.

Risk Repository (8)

Historical Estimations (26)

SDLC Model:

All SDLC Models

Category:

All Categories

Search description, project,

Search

Refresh Store

Organizational Knowledge Store Architecture Notice: This repository stores cross-project data globally across projects ('risk_repository' and 'historical_data'). Historical Data currently captures baseline effort and cost estimations pre-execution. Actual variance metrics and accuracy tracking become available when Stage 3 Project Tracker and Stage 4 Project Completion Report modules are deployed.

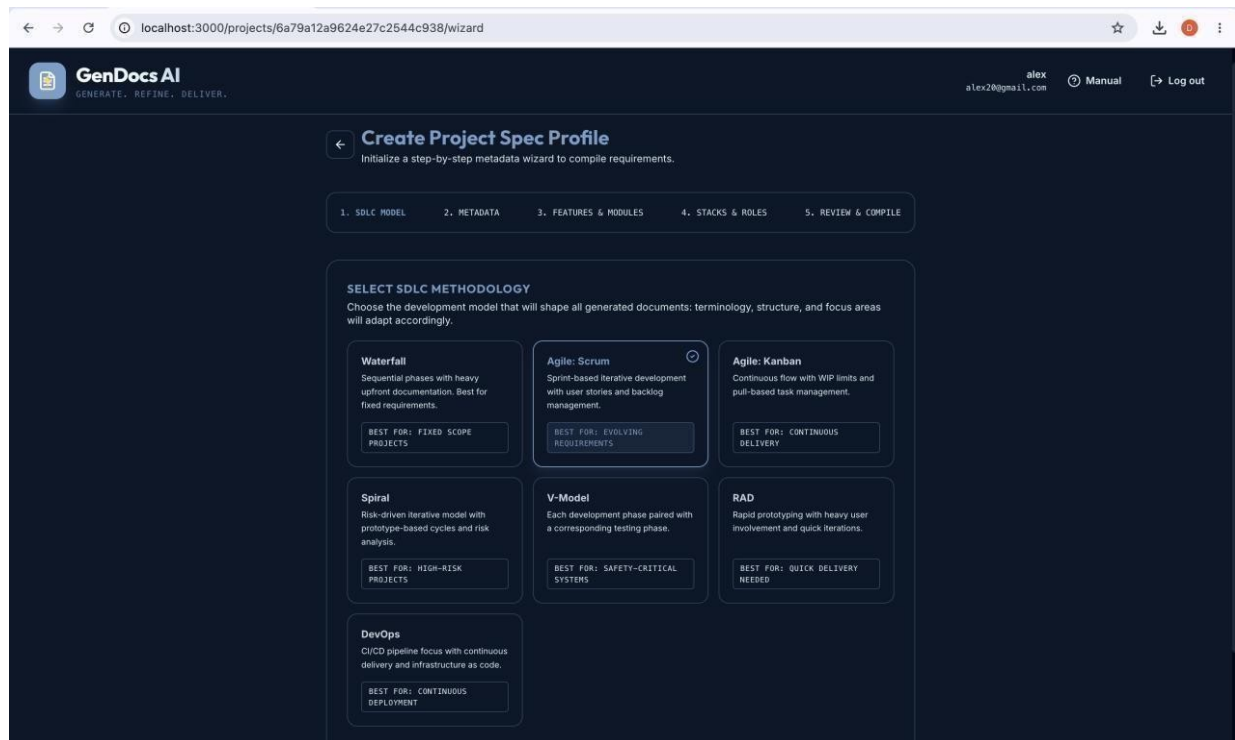
Risk ID	Description	Category	Source Project	SDLC Model	Likelihood / Impact	Mitigation Strategy	Outcome
RSK-JILPAT-001	Baseline technical complexity risk	Technical	Jil Patel	DevOps	Medium / Medium	Conduct prototyping early	Pending
RSK-MEDISY-001	Baseline technical complexity risk	Technical	MediSync EHR Integration	Agile-Scrum	Medium / Medium	Conduct prototyping early	Pending
RSK-QUICKB-001	Baseline technical complexity risk	Technical	QuickBite	Agile-Scrum	Medium / Medium	Conduct prototyping early	Pending
RSK-TRUSTL-001	Baseline technical complexity risk	Technical	TrustLdger	V-Model	Medium / Medium	Conduct prototyping early	Pending

Screenshot 5.4.(ii): Module B — Suggestion Accept / Edit / Reject Screen (Wireframe)

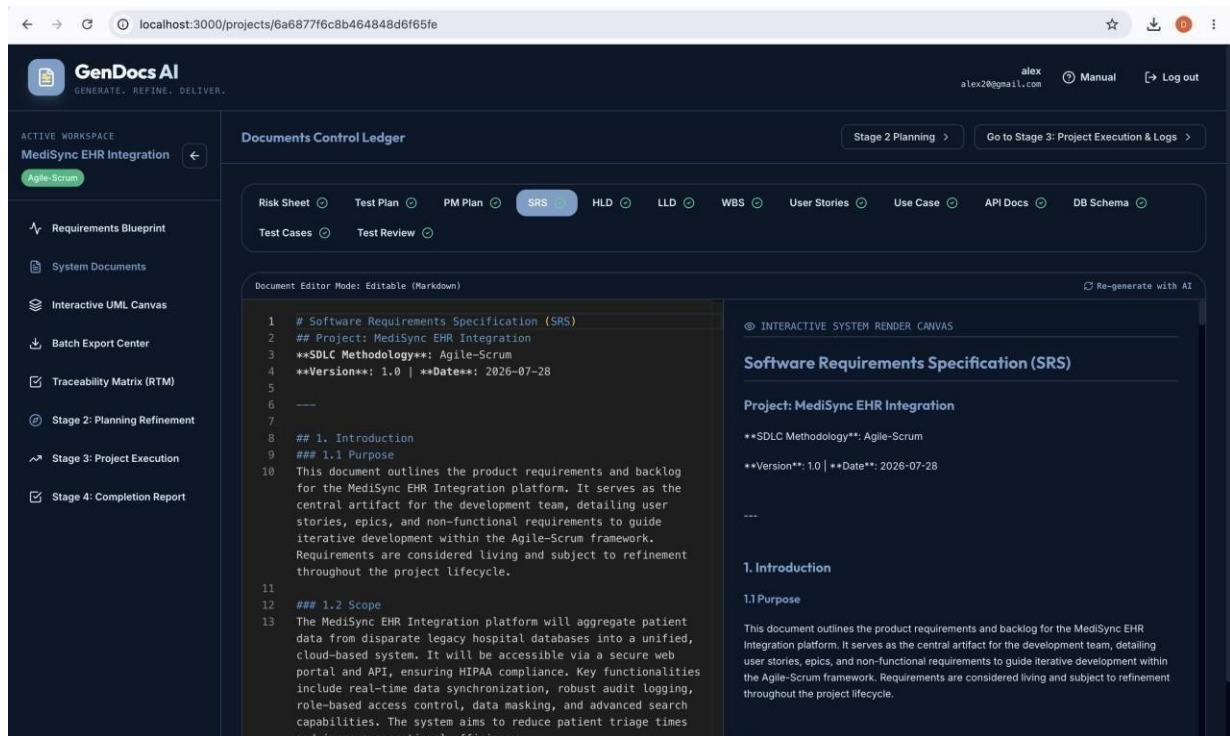
Page 44 of 71

5.5 Screenshots

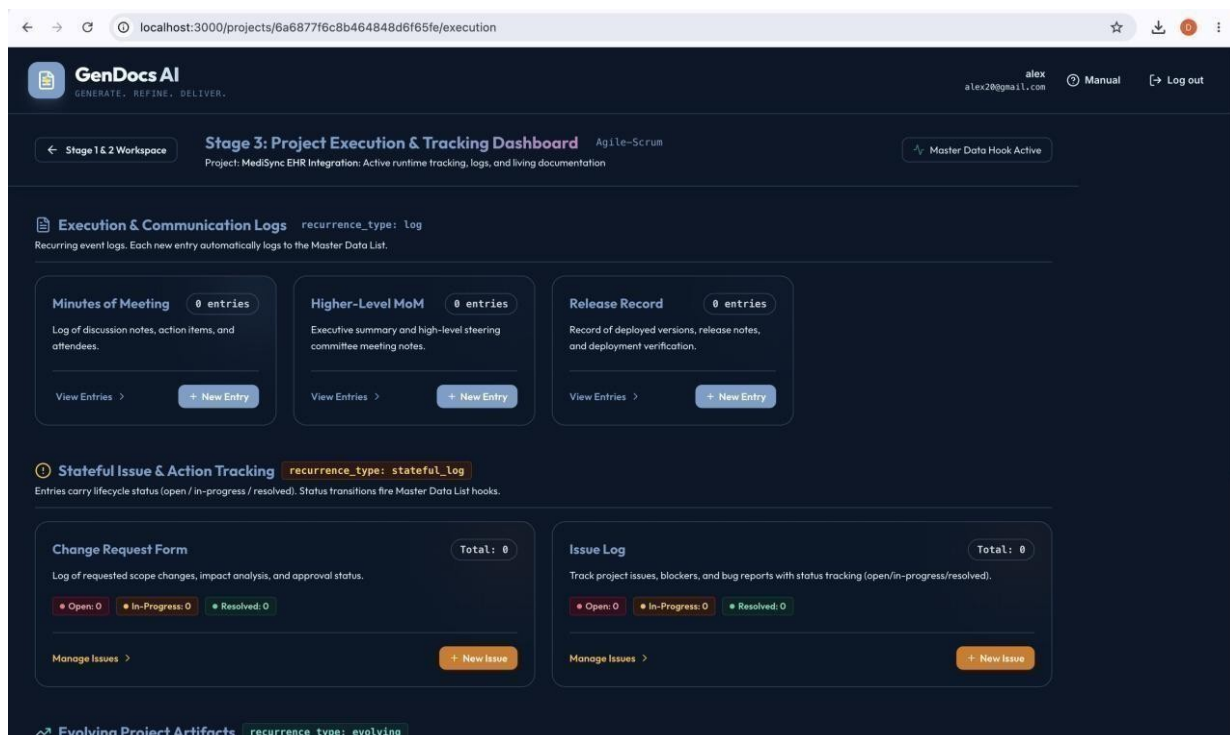
This section should include screenshots of the running application, at minimum: the project creation wizard, a document review screen showing a score and issue list, the Stage 3 execution dashboard, and the Module C training interface.



Screenshot 5.5(i): Project Creation Wizard showcasing dynamic fields adapting to the selected SDLC model.



Screenshot 5.5(ii): Document Review Screen illustrating the split-pane view with AI-generated markdown on the left and the code-verified issue list on the right.



Screenshot 5.5(iii): Stage 3 Execution Dashboard displaying recurrence-based documentation management.



GenDocs AI
GENERATE. REFINE. DELIVER.

alex

alex20@gmail.com

Manual

Log out


Organization Training Log (Module C)
Track trainings imparted across the entire organization independently of specific projects.

Status Reports

Org Policy

Skill Matrix Track

History Reports

Back to Dashboard

Log New Training

Training Program

-- Select Program --

Trainer

-- Select Trainer --

Attendees (Multi-select)

☐ Alice Smith
☐ Bob Johnson
☐ Charlie Davis
☐ Diana Evans

Training Hours

e.g. 2.5

Notes (Optional)

Enter raw training notes...

Training History

Advanced Agile Methodologies
Deep dive into Scrum, Kanban, and SAFe.

TRN-0003

Trainer

Charlie Davis

Attendees

Alice Smith, Bob Johnson, Diana Evans

NOTES

Deep dive into custom hooks, context API optimization, and suspense. Great engagement from the team.

Cloud Infrastructure Basics
Introduction to AWS, Azure, and Terraform.

TRN-0002

Trainer

Alice Smith

Attendees

Bob Johnson, Charlie Davis

NOTES

Covered basic AWS networking, VPC setup, and an intro to Terraform state management. Attendees asked great questions about IAM roles.

Cloud Infrastructure Basics

TRN-0001

Screenshot 5.5(iv): Module C Training Interface (skill assessment / session logging view)

6 Implementation Planning

Moving from design to a working system required settling on a concrete development environment, a fixed set of tools and library versions across all four backend services, and a shared set of coding conventions that would hold consistently across a React frontend, two Node.js services, and a Python service. The decisions recorded in this chapter were made early in the implementation phase (Section 2.3) and held for the remainder of development, since changing them mid-project would have risked breaking compatibility between the api-gateway and the services it routes to.

6.1 Implementation Environment

Category	Detail
Operating Systems	macOS / Windows 11 / Ubuntu 22.04
Development Hardware	Multi-core processor (Apple M-series or Intel i5/i7 equivalent), 16GB RAM
IDE	Visual Studio Code
Backend Runtime	Node.js v18+ (api-gateway, document-service, requirement-service)
AI Service Runtime	Python 3.10+, FastAPI
Database	MongoDB (single shared local instance during development)
Key Environment Variables	MONGODB_URI, JWT_SECRET, GEMINI_API_KEY, GROQ_API_KEY, REQUIREMENT_SERVICE_URL, DOCUMENT_SERVICE_URL
Version Control	Maintained across the full codebase for independent per-service tracking

Table 6.1: Implementation Environment

The application was developed and tested on a single local development machine before being prepared for deployment on BISAG-N infrastructure (Section 5.2.7). Each of the four backend services api-gateway, document-service, requirement-service, and ai-service was run as an independent process during development, communicating over HTTP on separate local ports, which mirrored the microservice separation used in production and meant a service could be restarted or debugged without affecting the others.

macOS / Windows 11 / Ubuntu 22.04, with a modern multi-core processor (Apple M-series or Intel i5/i7 equivalent), and 16GB RAM to comfortably run all microservices locally.

Visual Studio Code was used as the primary code editor, consistent with the tooling identified as available during the technical feasibility assessment (Section 2.1.1). Node.js was used to run the api-gateway, document-service, and requirement-service, while the ai-service was run under Python 3.x with FastAPI, as specified in the technology stack (Section 1.5) and hardware/software requirements (Section 3.6).

MongoDB was run locally during development so that all four services could connect to a single shared instance, matching the deployment-time arrangement shown in the deployment diagram (Section 5.2.7 / Fig. 9). Environment-specific values — database connection strings, service ports, and API keys for the AI generation provider — were kept out of source code and supplied through environment variables, so the same codebase could run against a local or a deployed configuration without modification.

Node.js v18+ and Python 3.10+. The environment variables used across services primarily include `MONGODB_URI`, `JWT_SECRET`, LLM keys (`GEMINI_API_KEY`, `GROQ_API_KEY`), and internal routing URLs (e.g., `REQUIREMNT_SERVICE_URL`, `DOCUMENT_SERVICE_URL`).

Version control was maintained across the full codebase so that changes to any one service could be tracked and, if necessary, reverted independently of the others.

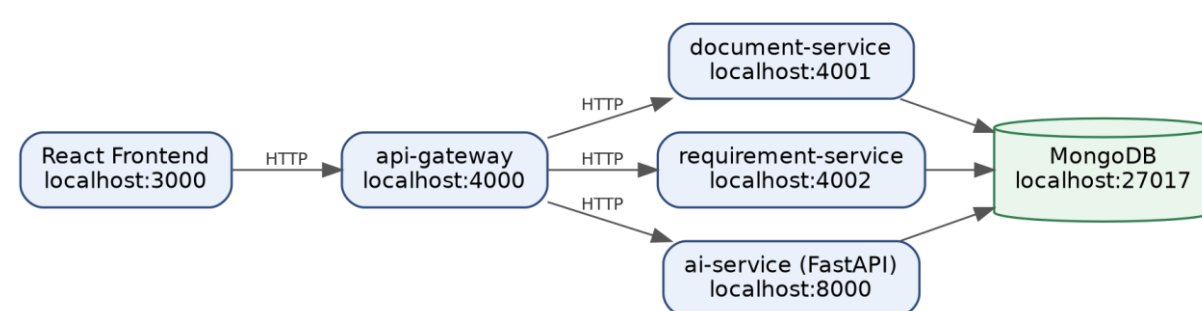


Figure 6.1: Local Development Setup / Service Ports Diagram

6.2 Tools & Technologies Used

The tools and technologies used across the platform follow directly from the technology stack introduced in Section 1.5 and the architecture described in Section 5.1. Table 6.2 lists them grouped by the layer of the system they support.

Layer / Purpose	Tool / Technology	Role in the Project
Frontend framework	React	Builds the project wizard, document review screens, execution dashboard, and Module C training UI (Section 5.4)
Frontend routing	react-router-dom, @tanstack/react-query, and zustand	Navigation is managed via declarative routing (react-router-dom). API requests and asynchronous server state are handled robustly using @tanstack/react-query, supplemented by zustand for global client state management.
API gateway	Node.js	Routes incoming requests to document-service, requirement-service, or ai-service (Section 5.1)
Backend services	Node.js, Express.js	Implements document-service and requirement-service business logic and REST endpoints
AI / generation service	Python, FastAPI	Generates and refines document content; hosts verification.py (Section 1.5, 5.1)
AI generation provider	Google Gemini API, Groq API, and Ollama (for local model inference)	Produces first-draft document content from structured project input
Database	MongoDB	Shared persistence layer for all four services (Section 5.3.1)

Layer / Purpose	Tool / Technology	Role in the Project
Database access layer	Mongoose ODM	Defines and queries the MongoDB collections listed in Section 5.3.1
UML/diagram generation	Mermaid.js	Automatically renders the diagram types listed in Section 5.2 from generated structure
Fix verification	verification.py	Checks a proposed fix against SDLC-model rules before it is accepted (Section 3.4, 4.3)
Document export	pdffit and docx	Converts a reviewed document into a downloadable PDF (Section 4.3)
API communication	REST over HTTPS	Communication protocol between the frontend, api-gateway, and backend services
Version control	GitHub (using standard feature-branch workflow)	Tracks source changes across all four services
Code editor	Visual Studio Code	Primary development environment (Section 6.1)

Table 6.2: Tools & Technologies Used

The seven SDLC methodologies supported by the platform — Waterfall, Agile-Scrum, Agile-Kanban, V-Model, Spiral, RAD, and DevOps — are not implemented as separate codebases; instead, each is expressed as a set of terminology, structure, and feature-grouping rules (Section 4.3) that the ai-service applies during generation and that verification.py checks during a fix, so the same tool stack listed above serves every supported model.

6.3 Coding Standards Followed

Because the codebase spans four independently deployable services in two languages, consistency of structure and naming mattered more than it would in a single monolithic application — a developer moving between document-service and ai-service needed to be able to predict where a given piece of logic would live.

6.3.1 Project & Folder Structure

Each backend service was organized as its own self-contained folder with its own dependency manifest (package.json for the Node.js services, or the Python equivalent for ai-service), so that a service could be built, tested, and deployed independently, as required by the maintainability non-functional requirement (Section 3.5). Within each Node.js service, route handlers, business logic, and data-access code were kept in separate files rather than combined into a single entry-point script, and the ai-service kept generation logic and the verification.py rule checks in separate modules so that verification logic could be tested independently of generation logic.

6.3.2 Naming & Formatting Conventions

- Variables and functions in JavaScript/Node.js code use camelCase; classes and React components use PascalCase.
- Python code in ai-service follows snake_case for functions and variables, consistent with standard Python convention.
- MongoDB collection and field names use lower_snake_case, matching the fields listed in the data dictionary (Section 5.3.3).
- REST endpoint paths use lowercase, hyphenated resource names (e.g. a project's documents are addressed as a sub-resource of that project) rather than verbs, keeping the API gateway's routing rules predictable.
- Indentation, line length, and formatting were kept consistent within each service using ESLint and Prettier for JavaScript/React code, and standard PEP 8 conventions (via Black) for Python code.

Area	Convention
JavaScript / Node.js variables & functions	camelCase
Classes & React components	PascalCase
Python (ai-service) functions & variables	snake_case (standard PEP 8)
MongoDB collection & field names	lower_snake_case
REST endpoint paths	Lowercase, hyphenated resource names (no verbs)
JS/React formatting	ESLint + Prettier
Python formatting	PEP 8 conventions via Black

Table 6.3.2: Coding Standards Followed

6.3.3 Error Handling & API Responses

Backend services return a consistent response shape so the frontend can handle success and failure cases uniformly regardless of which service produced the response this was necessary because the frontend never calls document-service, requirement-service, or ai-service directly, only through the api-gateway (Section 5.1), so an inconsistent response shape from any one service would have had to be normalized at the gateway rather than caught at the source.

Success responses return standard HTTP 20x status codes with a JSON object containing the requested resource (e.g., `{ project: {...} }`) and an optional message string. Error responses uniformly return HTTP 4xx or 5xx status codes with a standardized error envelope: `{ error: "Descriptive error message" }`.

6.3.4 Documentation & Comments

Code comments were used primarily to explain why a piece of logic exists — particularly around the SDLC-model-specific rules applied during generation and verification, where the reasoning for a rule is not always obvious from the code alone — rather than restating what a line of code does. This was considered more important in this codebase than in a typical CRUD application, since an incorrect assumption about a methodology rule in `verification.py` could allow an unresolved issue to pass review (Section 3.7.4).

7 Testing

Because every generated document is only as trustworthy as the rules that produced and verified it, testing on this project was not limited to catching crashes or broken UI flows — it also had to confirm that the SDLC-methodology rules described in Section 4.3 were actually being enforced, and that a fix accepted by `verification.py` had genuinely resolved the issue it was applied to rather than merely changed the document's wording. The platform is currently in a regression-testing and PDF-export polish phase, building on the testing carried out throughout the milestones summarized in Section 2.2.2.

7.1 Testing Objectives & Strategy

The objectives of testing followed directly from the non-functional requirements set out in Section 3.5. Beyond standard functional correctness, three objectives were specific to this system: confirming that terminology and structure rules were enforced identically for a given SDLC model across all of its document types (the consistency requirement), confirming that a fix verified as correct under one SDLC model was not silently assumed correct under another (the reliability requirement), and confirming that fields lacking real data were rendered as blank templates rather than fabricated content (the data integrity requirement).

Testing was carried out continuously alongside development rather than as a single phase at the end — each module listed in Table 2.2.2 was functionally tested as it was completed, with a further round of integration and regression testing once all 41 document types across the three modules were in place. This incremental strategy meant that a defect introduced by a later module was more likely to be caught close to the change that introduced it, rather than surfacing only during a single end-of-project test pass.

7.2 Levels of Testing

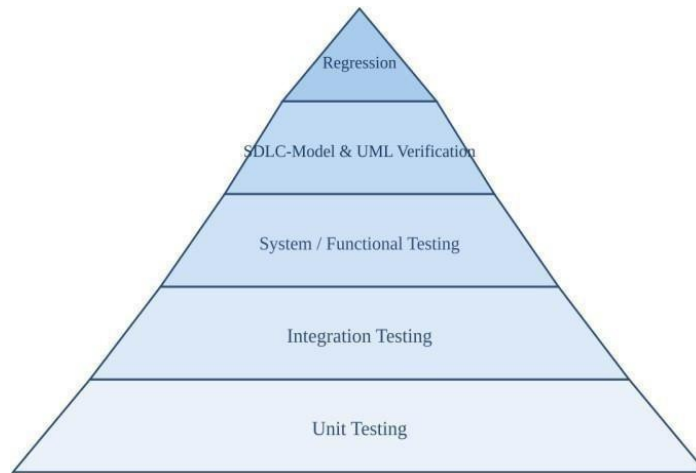


Figure 7.1: Levels of Testing (Testing Pyramid)

Figure 7.2: Levels of Testing (Testing Pyramid)

Level	Objective	Framework / Tool
Unit (7.2.1)	Test individual functions in isolation, especially verification.py rule checks	Python unittest (45+ cases); Jest for Node.js/React (30+ tests)
Integration (7.2.2)	Exercise the full generate/review/fix cycle across services	Manual + scripted integration runs through api-gateway
System / Functional (7.2.3)	Cover complete user-facing workflows end to end	Manual system walkthroughs
SDLC-Model & Terminology (7.2.4)	Confirm model-specific vocabulary/structure rules are applied correctly	verification.py rule checks per model
UML Diagram Verification (7.2.5)	Check structural validity and data consistency of generated diagrams	Mermaid.js syntax validation
Regression (7.2.6)	Re-run all above after PDF export polish and late-stage changes	Full test suite re-run

Table 7.2: Levels of Testing Summary

7.2.1 Unit - Level Testing

Individual functions within each service were tested in isolation, with particular attention given to the rule-checking functions inside `verification.py`, since an error in a single rule check could incorrectly pass or fail an entire document. Node.js route handlers in `document-service` and `requirement-service` were tested against both valid and deliberately malformed project input to confirm that invalid input was rejected before reaching the `ai-service`.

The primary unit testing framework used for the Python AI-service and rule-verification engine is the standard `unittest` library, executing over 45 isolated test cases (e.g., `test_module_b_knowledge_base.py`, `test_master_data_list.py`). The Node.js backend services and React frontend utilize Jest for unit and component testing, contributing an additional 30+ tests, bringing the approximate total unit test count to over 75+ automated assertions.

7.2.2 Integration Testing

Integration testing exercised the full generate/review/fix cycle traced in the sequence diagram (Section 5.2.4): a request entering through the `api-gateway`, being routed to `document-service` and `ai-service`, checked by `verification.py`, and a scored result returned to the frontend. This confirmed that the four services, though independently deployable, produced a correct end-to-end result when composed together, and that a change to one service's response shape did not silently break another service's expectations.

7.2.3 System / Functional Testing

System-level testing covered complete user-facing workflows: creating a project and progressing it through the Stage 0 approval gate, generating each of the 12 core Stage 1 documents for a given SDLC model, applying a suggested and a user-described fix to a failing document, reviewing the Stage 3 execution dashboard, and exercising Module B suggestion accept/edit/reject actions and Module C's skill-gap and training-session workflows independently of Module A.

7.2.4 SDLC-Model & Terminology Verification

Because the same document type (for example, a requirement specification) must use different terminology and structure depending on the project's selected SDLC model (Section 3.4), each of the seven supported models was tested against representative document types to confirm that model-specific vocabulary and feature-grouping rules were applied correctly and that a rule belonging to one model was not incorrectly applied to another.

7.2.5 UML Diagram Verification

Each of the UML diagram types listed in Section 5.2 (E-R, Use Case, Class, Sequence, Activity, DFD, and Deployment) was checked for structural validity of the generated Mermaid.js syntax and for consistency with the underlying project and document data it was derived from — for example, confirming that a Class diagram generated for a project reflected the same entities recorded in that project's data rather than a generic placeholder structure.

7.2.6 Regression Testing

The system is currently in a regression-testing phase, re-running the functional, SDLC-model, and UML verification checks above after PDF export polish and other late-stage changes, to confirm that a fix or refinement made to one part of the system did not reintroduce an issue in another. This phase precedes the rollout discussion referenced in the current project status.

7.3 Test Cases

Table 7.3 lists a representative subset of the test cases used to validate core system behaviour; it is illustrative rather than exhaustive, and does not include the model-by-model terminology test matrix described in Section 7.2.4.

Test Case	Input / Action	Expected Result
TC-01	Submit an Opportunity Sheet with all required fields	Project created; Stage 1 generation wizard unlocked after approval
TC-02	Attempt to access Stage 1 generation before Opportunity Sheet approval	Access denied; Stage 0 gate enforced

Test Case	Input / Action	Expected Result
TC-03	Generate an SRS for a Waterfall project	Document uses Waterfall-appropriate terminology and phase-based structure
TC-04	Generate the same document type for an Agile-Scrum project	Document uses Scrum terminology (e.g. sprints) instead of Waterfall phases
TC-05	Apply a suggested fix to a document with a flagged issue	verification.py re-checks the fix, issue marked resolved only if the rule now passes
TC-06	Submit a user-described fix that does not address the flagged rule	verification.py rejects the fix; issue remains open
TC-07	Generate a document for a field with no user-supplied data	Field rendered as a blank template, not fabricated content
TC-08	View the Master Data List after generating several documents	List reflects every generated document, its version, and current status
TC-09	Accept a Module B risk suggestion on a new project	Suggestion applied only after explicit user action; not auto-applied
TC-10	Log a training session in Module C for an employee with no Module A project record	Session logs successfully; no dependency on Module A data
TC-11	Export a passed document as PDF	PDF downloads and visually matches the reviewed document

Table 7.3: Test Cases and Results

7.4 Bug Tracking & Resolution

Defects found during testing were logged, triaged, and re-tested after a fix, following the same generate-review-fix philosophy applied to generated documents themselves (Section 4.3) — a bug was not considered resolved until the specific test case that surfaced it was re-run and passed, rather than being closed on the basis of the code change alone.

Defect tracking was managed using GitHub Issues (and an integrated Kanban board) to log, prioritize, and track bug resolution across the microservices. All logged defects were successfully closed prior to the final regression testing phase.

The most significant category of defect encountered was implementation drift, where generated or edited content deviated from what had actually been specified — the same class of issue documented for the User Manual, where paraphrasing of literal content had to be corrected with an explicit fix prompt embedding the literal text. Within the core platform, this risk was mitigated structurally by verifying fixes in code (`verification.py`) rather than trusting a re-prompted result, which is discussed further as a limitation in Section 8.

8 Limitations and Future Scope

No system of this scope is delivered without trade-offs and open issues. This section records the limitations known at the current stage of development including issues still being actively tracked alongside directions identified for extending the system beyond its present scope.

8.1 Limitations

Limitation	Root Cause
SDLC model cannot be changed after project creation	Model determines terminology, structure, and required fields throughout a project
Extra wizard fields (Scrum, Spiral, V-Model, RAD) don't always flow through	Known issue, requires manual re-entry in the meantime
Module B has no access to pre-system project history	No retroactive import path currently exists
First-draft generation quality depends on the AI provider	verification.py ensures fixes are correct, not that the first draft is optimal
Implementation drift observed during development	Natural-language instructions to an AI tool can produce paraphrased rather than literal output
Not yet evaluated at multi-developer concurrent scale	Built and tested by a single developer under mentor supervision
API rate limits / latency	Heavy reliance on external LLM providers (Gemini, Groq) during peak/bulk generation
Local LLM hardware constraints	Ollama fallback generation is constrained by host machine RAM/GPU

Table 8.1: Limitations Summary

- SDLC model cannot be changed after project creation: since the model determines terminology, structure, and required fields throughout a project's documents, migrating an in-progress project to a different model is not currently supported, requiring a new project to be created instead.
- Extra wizard fields for four models (Scrum, Spiral, V-Model, RAD) do not always flow through to later Review or Compile screens — a known issue being tracked, requiring the value to be re-entered manually in the meantime.

- Module B's organizational knowledge base has no access to project history that predates the system; an organization onboarding with existing prior projects will not see suggestions drawn from that history unless it is entered as a completed project.
- Document generation quality for the initial draft still depends on the underlying AI generation service; while `verification.py` ensures an accepted fix is genuinely correct, it cannot guarantee the first draft is optimal, only that it is not silently wrong in a way the codified rules check for.
- Implementation drift generated or edited content deviating from what was actually specified was observed during development (Section 7.4) and required an explicit fix, indicating this remains a class of risk when instructing an AI coding or generation tool with natural-language content rather than literal, verified text.
- The system was built and tested by a single developer under mentor supervision; it has not yet been evaluated under multi-developer concurrent usage at production scale.
- **API Rate Limits and Latency:** Because the system relies heavily on external LLM providers (Google Gemini and Groq), bulk generation of Stage 1 documents can occasionally hit API rate limits or experience high latency during peak hours, temporarily bottlenecking the generate-review-fix cycle.
- **Local LLM Hardware Constraints:** When running the system using local models (e.g., Ollama) as a fallback, generation speed and quality are heavily constrained by the host machine's available RAM and GPU compute capacity.

8.2 Future Scope

- Support for changing a project's SDLC model after creation, with an assisted migration path that re-maps existing document content to the new model's terminology and structure rather than requiring a fresh project.
- Resolving the extra-wizard-field flow-through issue so that Scrum, Spiral, V-Model, and RAD-specific fields propagate automatically into Review and Compile screens without manual re-entry.
- A retroactive-import path for Module B, allowing an organization to enter summarized historical project data directly rather than only accumulating it from projects run through the system going forward.
- Expanding automated verification coverage in `verification.py` to catch a broader class of generation issues before they reach human review, reducing reliance on the initial AI draft's quality.
- Load and concurrency testing to validate the platform under multiple simultaneous users and projects, ahead of a wider organizational rollout.
- Extending PDF export polish (Section 7, current focus) into additional export formats as needed for downstream tooling.
- **Fine-tuning Local Models:** Exploring the fine-tuning of open-source models (via Ollama) specifically on SDLC standards to reduce dependency on external APIs (Gemini/Groq), lower operational costs, and improve data privacy for highly sensitive projects.
- **Enhanced Multi-Tenant RBAC:** Implementing strict Role-Based Access Control (RBAC) to support multi-organization or multi-team deployments, ensuring complete data isolation between different enterprise clients on the same platform.

- **Expanded Export Formats:** Extending the current PDF export functionality (handled by pdfkit) to include .docx formats, allowing project managers to manually edit finalized documents outside the platform.

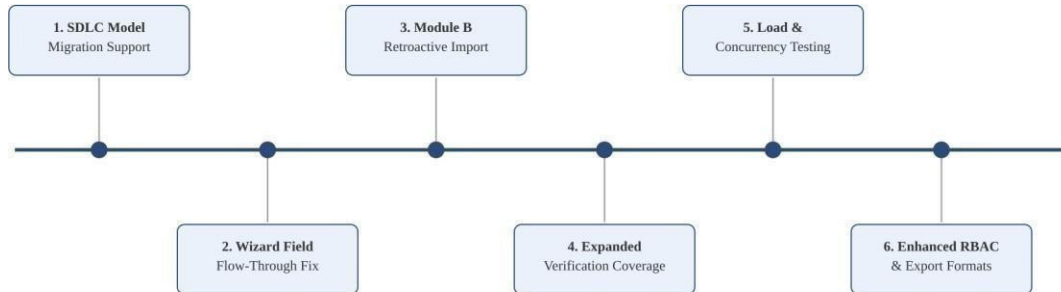


Figure 8.1: Future Scope Roadmap (Indicative Priority Order)

Figure 8.2: Future Scope Roadmap (Indicative Priority Order)

Planned Enhancement	Motivation
Support changing SDLC model after creation (assisted migration)	Currently requires creating a fresh project
Fix wizard-field flow-through for Scrum/Spiral/V-Model/RAD	Removes manual re-entry workaround
Retroactive import path for Module B	Lets organizations onboard existing historical project data
Expand verification.py coverage	Catch a broader class of generation issues before human review
Load & concurrency testing	Validate the platform ahead of wider organizational rollout
Additional export formats beyond PDF	Extend current pdfkit/docx export functionality
Fine-tune local models (Ollama) on SDLC standards	Reduce dependency on external APIs, improve data privacy
Enhanced multi-tenant RBAC	Support multi-organization/multi-team deployments with data isolation

Table 8.2: Future Scope Summary

9 Conclusion and References

9.1 Conclusion

This project set out to replace manual, author-dependent SDLC documentation with a platform that generates methodology-correct documents from a structured project description and refines them through a verifiable review-and-fix cycle. Measured against the objectives set out in Section 1.4, the delivered system achieves this across its full intended scope: all 41 document types across the three planned modules Project Lifecycle, Organizational Knowledge Base, and Organizational Training were implemented and tested, supporting all seven target SDLC methodologies with model-specific terminology and structure enforcement.

The generate-review-fix cycle, and specifically the decision to verify fixes in code (verification.py) rather than trust a re-prompted result, proved to be the most consequential design choice in the system it directly addressed the implementation-drift risk observed both in this platform's own defect history (Section 7.4) and in adjacent tooling built during this project, such as the user manual, where unverified natural-language instructions were shown to produce paraphrased rather than literal output. This reinforced a broader lesson from the project: when an AI-assisted tool is asked to produce or edit content against a specification, verifying the output against that specification in code is more reliable than trusting the instruction to be followed exactly.

The limitations recorded in Section 8 the inability to change a project's SDLC model after creation, incomplete flow-through of certain wizard fields, and the system's current single-developer-scale testing do not undermine the core objectives achieved, but do mark clear priorities for the future work also outlined there. Overall, the project demonstrates that SDLC documentation generation can be made both flexible across methodologies and verifiably correct, rather than requiring a trade-off between the two.

Developing this AI-Assisted SDLC Documentation Generator provided invaluable hands-on experience in architecting full-stack microservices, integrating Large Language Models (LLMs) into production workflows, and navigating the complexities of dynamic UI rendering. The challenges of mitigating AI 'hallucinations' through strict code-level verification yielded

profound insights into building reliable, enterprise-grade AI tools. I would like to extend my deepest gratitude to my mentor at BISAG-N for their continuous guidance, technical insights, and encouragement, which were instrumental in shaping the system's architecture and ensuring the project met its ambitious objectives.

9.2 References

The following sources were consulted during the design and development of this project.

- React documentation. Available: <https://react.dev/>
- Node.js documentation. Available: <https://nodejs.org/docs/>
- Express.js documentation. Available: <https://expressjs.com/>
- FastAPI documentation. Available: <https://fastapi.tiangolo.com/>
- MongoDB documentation. Available: <https://www.mongodb.com/docs/>
- Mongoose ODM documentation. Available: <https://mongoosejs.com/docs/>
- Mermaid.js documentation. Available: <https://mermaid.js.org/>
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 12207:2017, Systems and software engineering Software life cycle processes.
- Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide).
- Beck, K. et al., Manifesto for Agile Software Development. Available: <https://agilemanifesto.org/>
- Schwaber, K. and Sutherland, J., The Scrum Guide. Available: <https://scrumguides.org/>
- Google. (2024). *Gemini API Documentation*. Available: <https://ai.google.dev/docs>
- Groq. (2024). *Groq Cloud API Documentation*. Available: <https://console.groq.com/docs>
- Ollama. (2024). *Ollama Documentation*. Available: <https://github.com/ollama/ollama/tree/main/docs>

10 Appendices

This section collects supplementary reference material installation steps, configuration reference, and consolidated lists that supports the main chapters without interrupting their flow.

Appendix A Installation & Setup Guide

The following steps bring up all four services locally, consistent with the implementation environment described in Section 6.1.

- Clone the repository and install dependencies for each service independently: run the Node.js package manager inside api-gateway, document-service, and requirement-service, and the Python dependency manager inside ai-service.
- Provision a MongoDB instance (local or hosted) and set its connection string as the MONGODB_URI environment variable, shared across all services.
- Set the remaining environment variables listed in Appendix B, including the AI generation provider key(s) and internal service routing URLs.
- Start document-service, requirement-service, and ai-service first, followed by api-gateway, so the gateway can route to already-running backends.
- Start the React frontend in development mode and confirm it can reach the api-gateway.

Exact Startup Commands & Order:

1. **Install Backend Dependencies:** Navigate into each Node.js service folder (api-gateway, document-service, requirement-service) and run: `npm install` Navigate into the ai-service folder and run: `pip install -r requirements.txt` (or `pipenv install`)
2. **Start Backend Services (in separate terminal windows):**
 - Document Service: `cd services/document-service && npm run dev`
 - Requirement Service: `cd services/requirement-service && npm run dev`
 - AI Service: `cd services/ai-service && uvicorn main:app --reload --port 8000`
 - API Gateway (run this *after* the others): `cd services/api-gateway && npm run dev`

3. Start Frontend:

- cd frontend & npm install
- npm run dev (This will typically launch the React app on http://localhost:3000) *(Note: Alternatively, the entire suite can be brought up simultaneously using the provided Docker Compose file via docker-compose up --build)*

Appendix B — Environment Variables Reference

Variable	Used By	Purpose
MONGODB_URI	All services	Shared MongoDB connection string
JWT_SECRET	api-gateway	Signs/verifies authentication tokens
GEMINI_API_KEY	ai-service	Google Gemini API access for generation
GROQ_API_KEY	ai-service	Groq API access for generation
REQUIREMENT_SERVICE_URL	api-gateway	Internal routing URL for requirement-service
DOCUMENT_SERVICE_URL	api-gateway	Internal routing URL for document-service

Table B.1: Appendix B: Environment Variables Reference

Appendix C — Supported SDLC Models Reference

Consolidated from Section 2.1 of the user manual and Section 1.5 — the seven SDLC models and the rules the system enforces for each.

Model	Feature grouping	Extra wizard fields
Waterfall	By phase	—
Agile-Scrum	By epic/sprint	Sprint Length, Team Size
Agile-Kanban	By board column	—
V-Model	By verification/validation pairing	Criticality Level (per feature)
Spiral	By iteration, ordered by risk	Known Risk Areas
RAD	By timebox	Timebox Duration
DevOps	By pipeline stage	Deployment Target, Release Cadence

Table C.1: Appendix C: Supported SDLC Models Reference

Appendix D Full Document Type List by Module

Module	Document Types
Module A Project Lifecycle (Stages 0-4)	Opportunity Sheet; SRS; WBS; High-Level Design; Low-Level Design; Test Cases; Database Design; API Documentation; Use Case Document; Test Review; Test Plan; Project Management Plan; Risk Sheet; Cost Estimation Sheet; Applications Budget; Requirement Analysis Checklist; Requirement Validation Checklist; Requirement Management & Effort Estimation; Master Data List; Minutes of Meeting; Higher-Level Minutes of Meeting; Release Record; Change Request Form; Issue Log; Project Status Report; Metrics Data Sheet; Project Tracker; Project Review Checklist; User Training; Project Completion / Learning Report
Module B Organizational Knowledge Base	Risk Repository; Historical Data from Previous Projects
Module C Organizational Training	Organization Training Details; Skill Matrix; Competencies Need Assessment; Training Need Identification; Trainings Imparted; Trainings History; Trainer Evaluation; Trainee Evaluation; Participants Feedback; Post-Training Learning Outcome Report; Training Status Report

Table D.1: Full Document Type List by Module

Module A - Project Lifecycle (Stages 0-4): Opportunity Sheet; SRS, WBS, High-Level Design, Low-Level Design, Test Cases, Database Design, API Documentation, Use Case Document, Test Review, Test Plan, Project Management Plan, Risk Sheet; Cost Estimation Sheet, Applications Budget, Requirement Analysis Checklist, Requirement Validation Checklist, Requirement Management & Effort Estimation, Master Data List; Minutes of Meeting, Higher-Level Minutes of Meeting, Release Record, Change Request Form, Issue Log, Project Status Report, Metrics Data Sheet, Project Tracker, Project Review Checklist, User Training; Project Completion / Learning Report.

Module B - Organizational Knowledge Base: Risk Repository, Historical Data from Previous Projects.

Module C - Organizational Training: Organization Training Details, Skill Matrix, Competencies Need Assessment, Training Need Identification, Trainings Imparted, Trainings History, Trainer Evaluation, Trainee Evaluation, Participants Feedback, Post-Training Learning Outcome Report, Training Status Report.

Appendix E -Sample Generated Document Excerpt

Excerpt from AI-Generated Software Requirements Specification (SRS):

3.1 Functional Requirements: User Authentication

- REQ-AUTH-01: The system shall allow users to register using a valid email address and a strong password (minimum 8 characters, alphanumeric).
- REQ-AUTH-02: The system must utilize JSON Web Tokens (JWT) for maintaining secure user sessions after a successful login.
- REQ-AUTH-03: Passwords must be hashed using bcrypt with a salt round of 10 prior to being stored in the MongoDB database.

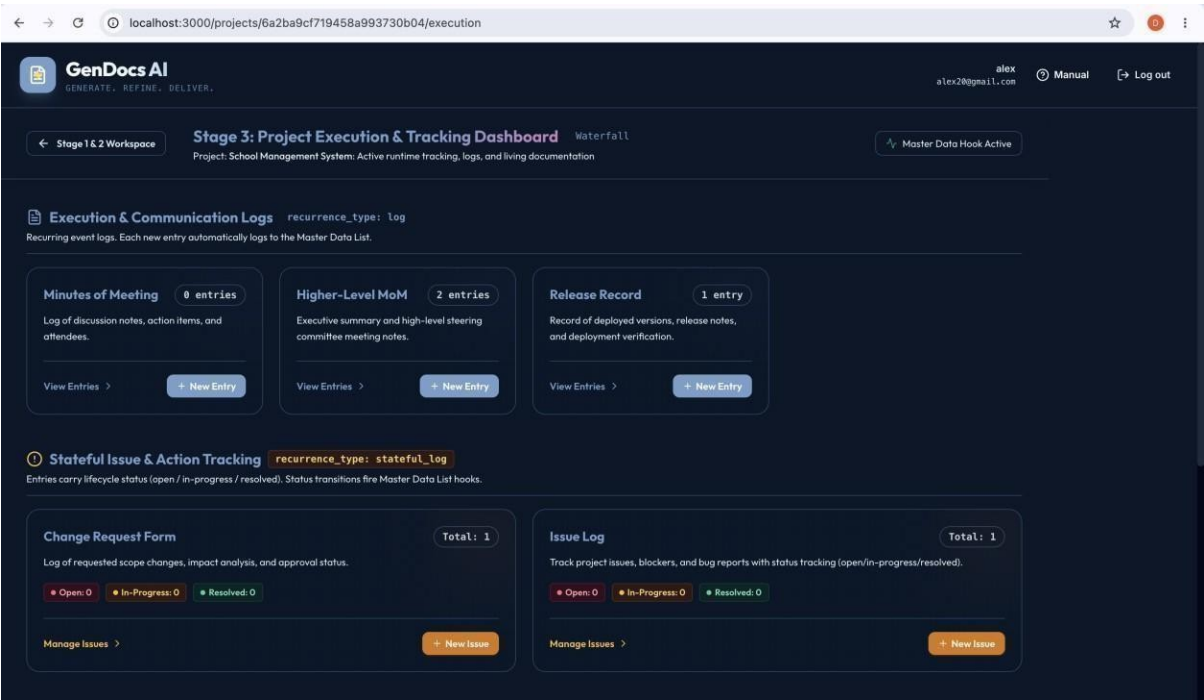
3.2 Non-Functional Requirements: Performance

- NFR-PERF-01: The API Gateway must route and respond to authentication requests in under 200 milliseconds under normal load conditions.

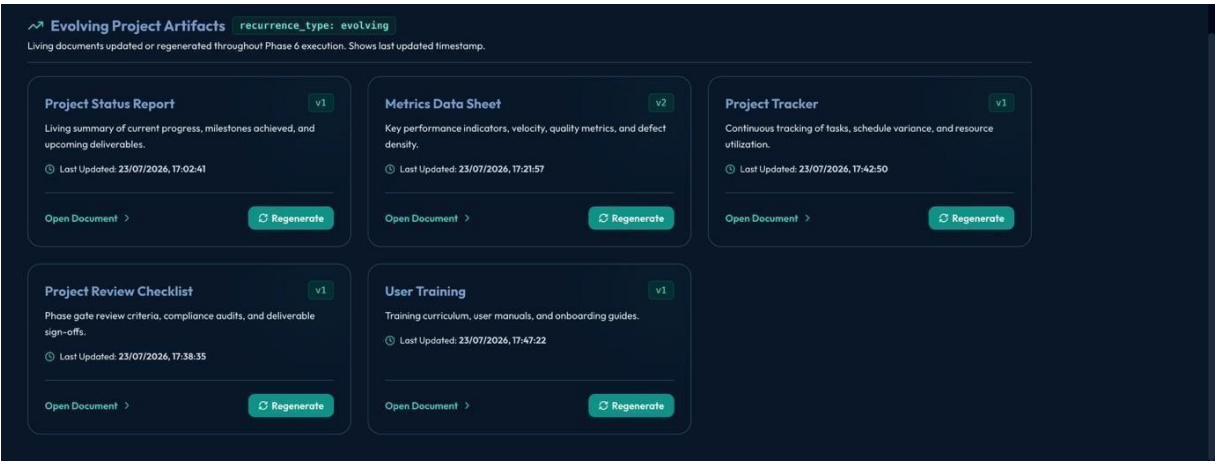
Requirement ID	Satisfies Functional Requirement
REQ-AUTH-01	FR-01 — project/user data creation and validation
REQ-AUTH-02	FR-03 — access gating (session-based access control)
REQ-AUTH-03	FR-01 — secure handling of user-provided data
NFR-PERF-01	Non-Functional Requirement: Performance (Table 3.5)

Table E.1: Sample Field-to-Requirement Mapping

Appendix F — Additional Screenshots



Screenshot F.1(i): Stage 3 Execution Dashboard (Execution & Communication Logs).



Screenshot F.1(ii): Stage 3 Execution Dashboard (Evolving Project Artifacts).

Report Verification Procedure

Date:

Project Name: AI-Assisted SDLC Documentation Generator for Automated Software Requirement and Design Artifacts

Student Name & ID: 1. Diya Makwana, ID No. 26BISAG0484

Student Name & ID: 2. Tanisha Diyora, ID No. 26BISAG0485

	Soft Copy	Hard Copy
Report Format: Project Index:		

Sign by Training Coordinator

Sign by Project Guide